

Roadmap de Domínio — TypeScript + JavaScript

Do runtime ao compile time: um guia prático para
desenvolvedores backend

DeepStudy PDF Generator

September 24, 2026

Contents

Como usar este documento	5
1 Introdução: compile time e runtime	7
2 Estratégia de estudo	9
2.1 Estimativa de tempo	10
2.2 Mapa: conceito de TS e o JavaScript que o explica	10
3 JavaScript essencial para dominar TypeScript	11
3.1 Scope e Lexical Scope	11
3.2 Closures	12
3.3 <code>this</code>	13
3.4 Objetos e referências	14
3.5 Funções	15
3.6 Arrays	15
3.7 Tratamento de erros	16
3.8 Optional chaining e nullish coalescing	18
3.9 Promises	18
3.10 Async/Await	19
3.11 Event Loop	19
3.12 Prototype	21
3.13 Prioridade média	21
3.13.1 Map, Set, WeakMap e WeakSet	22
3.13.2 ESM e CommonJS	22

3.13.3	Outros tópicos de prioridade média	23
3.14	Prioridade baixa	23
4	TypeScript — núcleo principal	25
4.1	Fundamentos	25
4.2	Classes em TypeScript	26
4.3	Narrowing	28
4.4	Generics	29
4.5	Utility Types	30
4.6	keyof, typeof e indexed access	32
4.7	Mapped Types	32
4.8	Conditional Types	33
4.9	infer	34
4.10	Template Literal Types	34
4.11	satisfies	35
4.12	unknown, any e never	36
4.13	Type Assertions	37
4.14	Function Overloads	37
5	TypeScript avançado	39
5.1	Structural Typing	39
5.2	Variance	40
5.3	Tipos recursivos, variadic tuples e type-level programming	41
5.4	Branded Types	41
5.5	Declaration files, module augmentation e declaration merging	42
5.6	Tipos de bibliotecas externas	43
5.7	Inferência avançada	43
5.8	Configuração do tsconfig	44
6	Ferramentas e ecossistema TypeScript	46
6.1	Execução e build	46

6.2	Lint e formatação	46
6.3	Testes e type tests	47
6.4	Validação em runtime (essencial)	47
6.5	Frameworks, banco de dados e configuração	48
7	TypeScript + JavaScript na prática	49
7.1	Closure + Generic	49
7.2	Promise + Generics	50
7.3	Type guard + runtime check	50
7.4	Classes + Prototype	51
7.5	async/await + Promise	51
7.6	EventEmitter + tipos	52
7.7	Callbacks + Generics	53
7.8	Quando usar X ou Y	54
7.9	Anti-patterns comuns	54
8	Projetos práticos por nível	55
9	Node.js que vale estudar depois	57
9.1	Streams com backpressure	58
9.2	Contexto por requisição e graceful shutdown	59
10	Ordem recomendada de estudo	60
11	Checklist de domínio	62
11.1	JavaScript	63
11.2	TypeScript	64
11.3	Ferramentas e ecossistema	64
11.4	Node.js	65
12	Regras do roadmap	66
A	Referências e Leitura Adicional	67

Contents	4
B Glossário	69
C FAQ e erros comuns	71
Index	73

Como usar este documento

Este documento **não é um livro didático**. É um **checklist de estudos e um mapa de prioridades** para desenvolvedores que já escrevem código e querem consolidar TypeScript como linguagem principal, entendendo o JavaScript que roda por baixo.

Ficha do roadmap

Foco	70% TypeScript, 30% JavaScript
Nível	pleno consolidando stack backend
Stack	Node.js, TypeScript, backend
Formato	checklist de estudos e mapa de prioridades
Duração sugerida	3 a 6 meses com dedicação consistente
Licença	CC BY-NC-SA 4.0

Como navegar

- Use o sumário para ir direto a uma seção.
- Marque os checkboxes do Capítulo [11](#) conforme dominar cada tópico.
- A ordem sugerida está no Capítulo [10](#), mas pode ser adaptada ao seu contexto.
- Cada tópico tem uma **marca de prioridade** e termina com uma caixa **Domínio**: um critério verificável para saber se você realmente aprendeu.

Legenda de prioridade

PRIORIDADE ALTA	Estude primeiro . Essencial para o dia a dia; sem isso o TypeScript vira um conjunto de regras decoradas.
PRIORIDADE MÉDIA	Estude depois de consolidar o essencial ou quando aparecer no projeto.
PRIORIDADE BAIXA	Útil, mas não bloqueia nada. Adie sem culpa e aprofunde sob demanda.

Pré-requisitos assumidos

- Git básico (commit, branch, merge);
- terminal/shell (navegação, variáveis de ambiente);
- Node.js instalado (versão LTS atual);
- editor configurado (VS Code recomendado, com o TypeScript do workspace);
- noções de HTTP e APIs REST;
- familiaridade com npm ou pnpm.

As seis perguntas que este documento responde

#	Pergunta	Onde
1	O que preciso saber de JavaScript para dominar TypeScript?	Cap. 3
2	O que devo estudar primeiro?	Caps. 3 e 10
3	O que posso deixar para depois?	Seções 3.14 e 5
4	Quais assuntos são especificamente de TypeScript?	Caps. 4, 5 e 6
5	Quais assuntos de Node.js complementam esse conhecimento?	Cap. 9
6	Como saber se realmente dominei cada tópico?	Cap. 11

Chapter 1

Introdução: compile time e runtime

A distinção fundamental que organiza todo este roadmap:

TypeScript existe principalmente em compile time, enquanto JavaScript é o que efetivamente executa em runtime.

O TypeScript é um sistema de tipos estático sobre o JavaScript. O compilador lê o código, verifica se os tipos são coerentes e depois **apaga os tipos** (*type erasure*). O que chega ao Node.js é JavaScript puro.

```
1 function pegarNome(usuario: { nome: string }) {  
2   return usuario.nome;  
3 }
```

Depois da compilação:

```
1 function pegarNome(usuario) {  
2   return usuario.nome;  
3 }
```

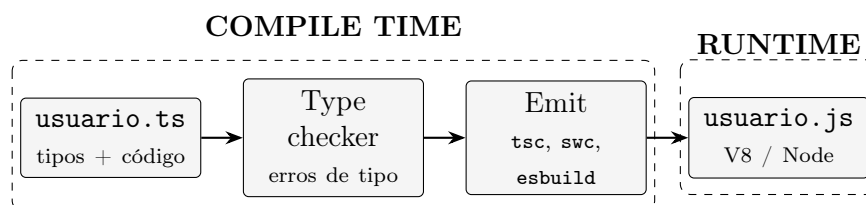


Figure 1.1: Os tipos existem até o emit. Tudo o que acontece depois, inclusive os bugs de runtime, é JavaScript sem tipos e sem garantias.

A consequência é direta: o TypeScript fornece **garantias estáticas**, mas **não altera o comportamento do JavaScript em runtime**. Ele não valida o body de uma requisição, não converte tipos e não impede que um `JSON.parse` devolva algo diferente do

declarado. Sem entender o comportamento real do JS (closures, `this`, prototype, Event Loop, coerção), você escreve TypeScript que *parece* correto e quebra em produção:

```
1 type Usuario = { nome: string };
2
3 const body = '{"nome": 42}'; // veio da rede
4 const usuario = JSON.parse(body) as Usuario; // "confie em mim, compilador"
5
6 usuario.nome.toUpperCase();
7 // compila sem erros -- e em runtime:
8 // TypeError: usuario.nome.toUpperCase is not a function
```

Modelo mental

Os tipos são uma **descrição** do que você espera que o JavaScript faça. Quando descrição e realidade divergem, o runtime vence. Por isso dominar TypeScript exige entender o JavaScript por baixo dele — e validar em runtime tudo o que vem de fora (Seção 6.4).

Recursos que geram código

Alguns recursos do TS *emitem* JavaScript: `enum`, `namespace` com valores, `parameter properties` (`constructor(private x: T)`) e `decorators`. A execução nativa de `.ts` pelo Node (*type stripping*) e a flag `erasableSyntaxOnly` rejeitam os que não podem ser simplesmente apagados.

Chapter 2

Estratégia de estudo

70% TypeScript	Foco principal, onde o tempo é investido: modelagem com tipos, narrowing, generics, tipos derivados, ferramentas e configuração.
30% JavaScript	Fundamento, estudado conforme sustenta o TS: escopo, closures, this , referências, erros, async, Event Loop, prototype.
0%	Detalhes históricos e quirks obscuros.

Você **não precisa** dominar todos os detalhes históricos ou obscuros do JavaScript para dominar TypeScript. Tabelas de coerção do `==`, `with` ou as idiossincrasias de `Date` não mudam como você modela tipos. O que muda é entender **de onde vêm os valores e como eles se comportam**.

Estratégia: aprender TypeScript e estudar JavaScript conforme ele explica o comportamento do código TypeScript.

Na prática: se você está estudando generics e surge uma dúvida sobre closures, pare e estude closures. A motivação vem do uso, não de uma trilha linear exaustiva.

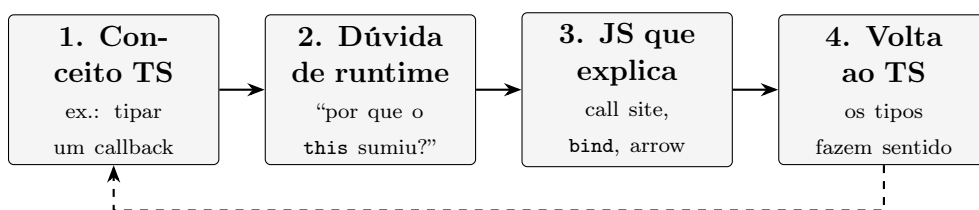


Figure 2.1: O ciclo de estudo: o TypeScript puxa o JavaScript que for necessário.

2.1 Estimativa de tempo

Bloco	Tempo estimado
JavaScript essencial	4–6 semanas
TypeScript: fundamentos até generics	6–8 semanas
TypeScript avançado	4–6 semanas
Ferramentas e ecossistema	2–3 semanas
Node.js internals	4–6 semanas
Prática contínua com projetos	em paralelo, sempre
Total sugerido	3 a 6 meses com dedicação consistente

Os blocos se sobrepõem: o total é menor que a soma porque JavaScript e TypeScript são estudados em paralelo.

2.2 Mapa: conceito de TS e o JavaScript que o explica

No TypeScript você estuda...	...o JS que explica é	Por quê
Tipos de funções, generics em callbacks	Funções de primeira classe, closures	O tipo descreve uma função que captura estado
<code>typeof</code> , <code>instanceof</code> , <code>in</code>	Operadores de runtime, prototype chain	Narrowing é o TS lendo checkagens reais
Classes, <code>private</code> , <code>#campo</code>	Prototype, class fields	<code>private</code> some no emit; <code>#</code> existe em runtime
<code>Promise<T></code> , <code>Awaited</code>	Promises, Event Loop	O tipo descreve o valor futuro, não quando ele chega
<code>readonly</code> , <code>Readonly<T></code>	Referências, mutabilidade	<code>readonly</code> é só compile time e é raso
<code>keyof</code> , indexed access	Objetos, propriedades dinâmicas	<code>obj[chave]</code> é JS; o TS tipa a chave
Parâmetro <code>this</code>	Regras de binding de <code>this</code>	O TS modela uma regra resolvida no call site
<code>catch (err: unknown)</code>	Tratamento de erros	Em JS qualquer valor pode ser lançado
<code>import type</code> , <code>module</code>	ESM e CommonJS	O <code>tsconfig</code> precisa refletir o runtime

Chapter 3

JavaScript essencial para dominar TypeScript

As seções 3.1 a 3.12 são **PRIORIDADE ALTA**: elas explicam praticamente todo bug de runtime que o TypeScript não pega. Estude na ordem. Os tópicos de prioridade média e baixa estão no fim do capítulo.

3.1 Scope e Lexical Scope

PRIORIDADE ALTA

- escopo global, de função e de bloco;
- `let`, `const` e `var` — diferenças reais;
- lexical environment e bindings;
- Temporal Dead Zone (TDZ) e hoisting.

Lexical scope significa que o escopo é decidido por *onde a função foi escrita*, não por onde ela é chamada. Cada bloco ou função cria um *lexical environment*: uma tabela de **bindings** (nome → valor) mais uma referência para o ambiente externo.

```
1 console.log(a); // undefined -- var é hoisted e inicializado com undefined
2 console.log(b); // ReferenceError -- b está na TDZ até a declaração
3 var a = 1;      // escopo de função (ou global)
4 let b = 2;      // escopo de bloco
5 const c = { n: 3 }; // o BINDING é constante -- o objeto continua mutável
6
7 if (true) {
8   var x = 'vaza'; // pertence à função/módulo, não ao bloco
9   let y = 'fica'; // pertence ao bloco
10 }
```

```

11 console.log(x); // 'vaza'
12 console.log(y); // ReferenceError: y is not defined
13 c.n = 4;        // ok
14 c = {};         // TypeError: Assignment to constant variable.

```

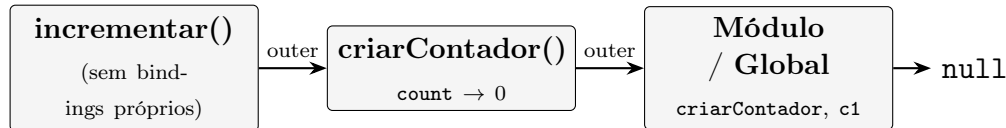


Figure 3.1: Cadeia de lexical environments. Para resolver `count` dentro de `incrementar()`, o JS procura no ambiente atual e sobe pela referência `outer` até encontrar; se chegar a `null`, lança `ReferenceError`.

Domínio: você explica a diferença entre TDZ e o hoisting de `var` e prevê o resultado de qualquer trecho com blocos aninhados sem executar.

3.2 Closures

PRIORIDADE ALTA

- conceito e por que existem;
- funções retornadas e estado privado;
- closures em loops e a diferença entre `let` e `var`;
- factory functions.

Uma **closure** é uma função mais o lexical environment em que ela foi criada. Se a função sobrevive à execução de quem a criou, o ambiente sobrevive junto. É o mecanismo por trás de estado privado e de factory functions.

```

1 function criarContador() {
2   let count = 0;           // ninguém de fora acessa diretamente
3   return {
4     incrementar: () => ++count,
5     atual: () => count,
6   };
7 }
8
9 const c1 = criarContador();
10 c1.incrementar();
11 c1.incrementar();
12 c1.atual(); // 2
13
14 const c2 = criarContador();
15 c2.atual(); // 0 -- cada chamada cria um NOVO ambiente

```

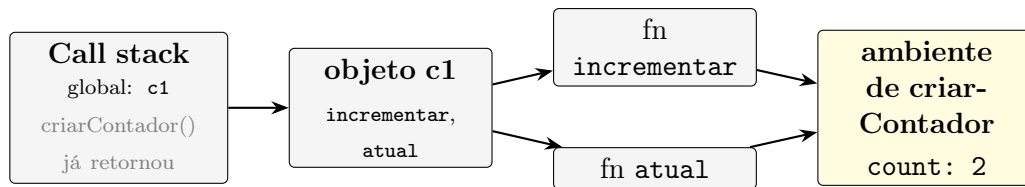


Figure 3.2: O frame de `criarContador()` saiu da pilha, mas o ambiente com `count` continua vivo no heap enquanto alguma função apontar para ele (as setas à direita são o link interno `[[Environment]]` de cada função).

A pergunta clássica sobre closures em loops:

```

1 for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
2 for (let j = 0; j < 3; j++) setTimeout(() => console.log(j)); // 0 1 2
3 // var: UM binding compartilhado pelo loop inteiro.
4 // let: um binding NOVO por iteração -- cada callback captura o seu.
  
```

Domínio: você escreve uma factory com estado privado (por exemplo, um rate limiter em memória) sem hesitar, explica o 3 3 3 e sabe que closures retêm memória enquanto forem referenciadas.

3.3 this

PRIORIDADE ALTA

O valor de `this` é decidido no **call site**, isto é, na forma como a função é chamada, e não onde ela foi definida. Arrow functions são a exceção: não têm `this` próprio e herdam o do escopo léxico.

```

1 const conta = {
2   saldo: 100,
3   ver() { return this.saldo; },
4   verArrow: () => this?.saldo, // this léxico do módulo -> undefined
5 };
6
7 conta.ver(); // 100 -- call site: conta.ver()
8 const solta = conta.ver;
9 solta(); // TypeError: this é undefined (strict mode)
10 solta.call({ saldo: 5 }); // 5
11 solta.apply({ saldo: 7 }, []); // 7 (argumentos como array)
12 const presa = conta.ver.bind(conta);
13 presa(); // 100
14 conta.verArrow(); // undefined -- arrow ignora o call site
  
```

Forma de chamar	Função normal	Arrow function
obj.f()	obj	this de onde foi definida
f()	undefined (strict) / global (sloppy)	idem
f.call(x) / f.apply(x, args)	x	ignora x
f.bind(x)()	x (fixo)	ignora x
new F()	novo objeto	erro: não é construtor

Bug real de backend

router.get('/pedidos', controller.listar) passa o método
solto: dentro dele, this.service é undefined. Soluções:
 controller.listar.bind(controller), (req, res) =>
 controller.listar(req, res) ou declarar o método como arrow property.

Domínio: dado qualquer call site, você diz o valor de `this`; sabe quando `bind` é necessário e quando *não* usar arrow functions (métodos de protótipo, construtores).

3.4 Objetos e referências

PRIORIDADE ALTA

Primitivos são copiados por valor; objetos, arrays e funções são manipulados por **referência**. Spread faz **shallow copy**: o primeiro nível é novo, os objetos aninhados continuam compartilhados.

```

1  const a = { nome: 'Ana', endereco: { cidade: 'POA' } };
2  const b = a;           // mesma referência: b === a
3  const c = { ...a };    // shallow copy (equivalente a Object.assign({},
   ↪ a))
4  c.nome = 'Bia';        // não afeta a
5  c.endereco.cidade = 'SP'; // AFETA a.endereco -- objeto aninhado
   ↪ compartilhado
6  const d = structuredClone(a); // deep copy (Node 17+)
7
8  Object.freeze(a);      // não permite adicionar, remover nem alterar (raso!)
9  Object.seal(a);        // permite alterar, mas não adicionar nem remover
10 a.endereco.cidade = 'RJ'; // funciona mesmo com freeze: freeze é raso
11
12 const { nome, endereco: { cidade }, ...resto } = a; // destructuring + rest
13 function salvar({ id, ...dados }) { /* ... */ }      // destructuring em
   ↪ parâmetro

```

Conexão com TypeScript

`readonly` e `Readonly<T>` são rasos e só existem em compile time. Outra referência sem `readonly` pode mutar o mesmo objeto. Para imutabilidade em runtime, use `Object.freeze` (também raso).

Domínio: você identifica mutação acidental por referência compartilhada (por exemplo, um objeto default reaproveitado entre requisições) e escolhe conscientemente entre `spread`, `Object.freeze` e `structuredClone`.

3.5 Funções

PRIORIDADE ALTA

Funções de primeira classe, callbacks, funções de ordem superior (HOF), parâmetros rest, default parameters e tagged templates.

```
1 const aplicar = (fn, ...args) => fn(...args); // rest na definição, spread na
  ↳ chamada
2
3 function saudar(nome = 'mundo') { // default dispara só para
  ↳ undefined
4   return `olá, ${nome}`;
5 }
6
7 const comLog = (fn) => (...args) => { // HOF: recebe e devolve função
8   console.log('chamando', fn.name, args);
9   return fn(...args);
10 };
11
12 // tagged template: a função recebe as partes fixas e os valores separados
13 function sql(partes, ...valores) {
14   return { texto: partes.join('?'), valores }; // base de libs de SQL seguras
15 }
16 sql`SELECT * FROM pedidos WHERE id = ${id} AND status = ${status}`;
17 // { texto: 'SELECT * FROM pedidos WHERE id = ? AND status = ?', valores: [id,
  ↳ status] }
```

Domínio: você escreve um wrapper de função (log, retry, cache) como HOF, sabe que default parameters não disparam para null e explica por que tagged templates evitam SQL injection.

3.6 Arrays

PRIORIDADE ALTA

Domine sem consultar `map`, `filter`, `reduce`, `find`, `findIndex`, `some`, `every`, `sort`, `flat`, `flatMap` e `at`, e saiba quais deles **mutam** o array original.

```

1  const pedidos = [
2    { id: 1, cliente: 'ana', total: 120, pago: true,  itens: ['a', 'b'] },
3    { id: 2, cliente: 'bia', total: 80,  pago: false, itens: ['c'] },
4    { id: 3, cliente: 'ana', total: 50,  pago: true,  itens: [] },
5  ];
6
7  const pagos      = pedidos.filter(p => p.pago);
8  const faturado   = pagos.reduce((soma, p) => soma + p.total, 0);    // 170
9  const porCliente = pedidos.reduce((acc, p) => {
10    (acc[p.cliente] ??= []).push(p);
11    return acc;
12  }, {});                // { ana: [...], bia: [...] }
13
14  pedidos.find(p => p.id === 2);           // o objeto ou undefined
15  pedidos.findIndex(p => p.id === 9);      // -1
16  pedidos.some(p => !p.pago);              // true
17  pedidos.every(p => p.total > 0);         // true
18  pedidos.flatMap(p => p.itens);           // ['a', 'b', 'c']
19  [[1, [2]], [3]].flat(Infinity);         // [1, 2, 3]
20  pedidos.at(-1);                        // último elemento
21
22  [10, 9, 1].sort();                     // [1, 10, 9]  <- compara como STRING
23  [10, 9, 1].sort((a, b) => a - b);       // [1, 9, 10]
24  // sort() muta; toSorted() (ES2023) devolve uma cópia
25
26  pedidos.forEach(p => enviar(p));         // efeito colateral, retorna
    ↪ undefined
27  const ids = pedidos.map(p => p.id);      // transformação, retorna novo array

```

Domínio: você escreve um `reduce` de agrupamento sem consultar nada, nunca chama `sort()` sem comparador em números, usa `map` para transformar e `forEach` só para efeitos, e sabe por que `find` retorna `T | undefined` no TypeScript.

3.7 Tratamento de erros

PRIORIDADE ALTA

- `Error` e subclasses nativas (`TypeError`, `RangeError`...);
- como estender `Error` corretamente e usar `cause`;
- `try / catch / finally`;
- erros em código assíncrono e `AggregateError`;

- quando relançar e quando engolir.

```

1 class ErroDeDominio extends Error {
2   constructor(message: string, readonly codigo: string, options?: ErrorOptions)
3     ↪ {
4     super(message, options);           // options.cause (ES2022) preserva o erro
5     ↪ original
6     this.name = new.target.name;      // 'ErroDeDominio' ou o nome da subclasse
7   }
8 }
9
10 class PedidoNaoEncontrado extends ErroDeDominio {
11   constructor(id: string) {
12     super(`pedido ${id} não encontrado`, 'PEDIDO_404');
13   }
14 }
15
16 try {
17   await repo.buscar(id);
18 } catch (err) {
19   if (err instanceof PedidoNaoEncontrado) {
20     return res.status(404).send({ codigo: err.codigo }); // erro esperado:
21     ↪ tratar
22   }
23   throw new ErroDeDominio('falha ao buscar pedido', 'INTERNO', { cause: err });
24   ↪ // relançar
25 } finally {
26   metricas.registrar('buscar_pedido'); // roda com ou sem erro
27 }

```

Armadilhas

- Com target ES5, instanceof em subclasses de Error quebra; era preciso `Object.setPrototypeOf(this, new.target.prototype)`. Com ES2015+ não é mais necessário.
- Em JavaScript qualquer valor pode ser lançado (`throw 'x'`); por isso o TS tipa `err` como `unknown`.
- Um `try` sem `await` não captura a rejeição de uma Promise. Uma rejeição sem tratamento (`unhandledRejection`) derruba o processo no Node moderno.
- `Promise.any` rejeita com `AggregateError`, que carrega todos os erros em `.errors`.
- Engula um erro apenas quando ele é esperado e você sabe o que fazer; caso contrário, relance com contexto.

Domínio: você cria erros customizados com `name`, `código` e `cause`, distingue erros esperados (4xx) de inesperados (5xx) e nunca deixa uma Promise rejeitar sem tratamento.

3.8 Optional chaining e nullish coalescing

PRIORIDADE ALTA

```

1 const cidade = usuario?.endereco?.cidade; // undefined se algo no caminho for
  ↳ null/undefined
2 const retorno = hooks.onSalvar?.(pedido); // chama só se a função existir
3 const item = lista?.[0];
4
5 const limite = config.limite ?? 100; // só null/undefined caem no
  ↳ default
6 const limite2 = config.limite || 100; // 0, '' e false TAMBÉM caem --
  ↳ bug se 0 é válido
7
8 opcoes.timeout ??= 5000; // atribui só se null/undefined
9 cache.dados ||= []; // atribui se falsy
10 flags.ativo &&= usuarioPodeAtivar; // atribui só se truthy

```

Domínio: você escolhe entre ?? e || pelo significado de 0, '' e false no seu domínio, e não usa ?. para esconder um valor que deveria existir.

3.9 Promises

PRIORIDADE ALTA

Uma Promise tem três estados — *pending*, *fulfilled* ou *rejected* — e muda de estado uma única vez. `then` sempre retorna uma **nova** Promise, o que permite o encadeamento. `Promise.resolve(v)` e `Promise.reject(e)` criam promises já resolvidas ou rejeitadas.

```

1 buscarUsuario(id)
2   .then(u => buscarPedidos(u.id)) // retornar Promise "achata" a
  ↳ cadeia
3   .then(pedidos => pedidos.length)
4   .catch(err => { log(err); return 0; }) // recupera: a cadeia segue
  ↳ fulfilled
5   .finally(() => fecharConexao()); // roda sempre; não recebe o valor

```

Combinador	Resolve quando	Rejeita quando	Uso típico
<code>Promise.all</code>	todas resolvem	na primeira rejeição	dados obrigatórios em paralelo
<code>Promise.allSettled</code>	todas terminam	nunca rejeita	lote tolerante a falhas
<code>Promise.race</code>	a primeira terminar	a primeira a terminar rejeitar	timeout
<code>Promise.any</code>	a primeira resolver	todas rejeitam (<code>AggregateError</code>)	fallback entre réplicas

Domínio: você escolhe o combinador certo sem pensar, usa `allSettled` quando falhas parciais são aceitáveis e sabe que `Promise.all` não cancela as outras promises quando uma falha.

3.10 Async/Await

PRIORIDADE ALTA

`async/await` é açúcar sobre Promises: a função *sempre* retorna uma Promise, e `await` suspende apenas aquela função (não a thread), devolvendo o controle ao Event Loop.

```
1 // sequencial: ~300ms + ~300ms
2 const usuario = await buscarUsuario(id);
3 const pedidos = await buscarPedidos(id);
4
5 // paralelo: ~300ms -- as duas já começaram antes do await
6 const [u, p] = await Promise.all([buscarUsuario(id), buscarPedidos(id)]);
7
8 try {
9   await salvar(pedido);
10 } catch (err) {
11   // captura a rejeição do await acima
12 }
13
14 // armadilha: forEach não espera callbacks async
15 itens.forEach(async (i) => await salvar(i)); // dispara tudo e segue em frente
16 for (const i of itens) await salvar(i);      // sequencial de verdade
17 await Promise.all(itens.map(salvar));        // paralelo de verdade
18
19 // async iteration: consome fontes assíncronas item a item
20 for await (const linha of lerLinhas('grande.csv')) { /* ... */ }
```

Domínio: você identifica awaits sequenciais que deveriam ser paralelos, sabe que `return promise` sem `await` dentro de `try` escapa do `catch` e explica por que `forEach` com `async` é um bug.

3.11 Event Loop

PRIORIDADE ALTA

Call stack, microtasks, macrotasks, timers, I/O, `process.nextTick`, `setImmediate` e a relação com Node.js e libuv.

Preveja a saída antes de ler os comentários:

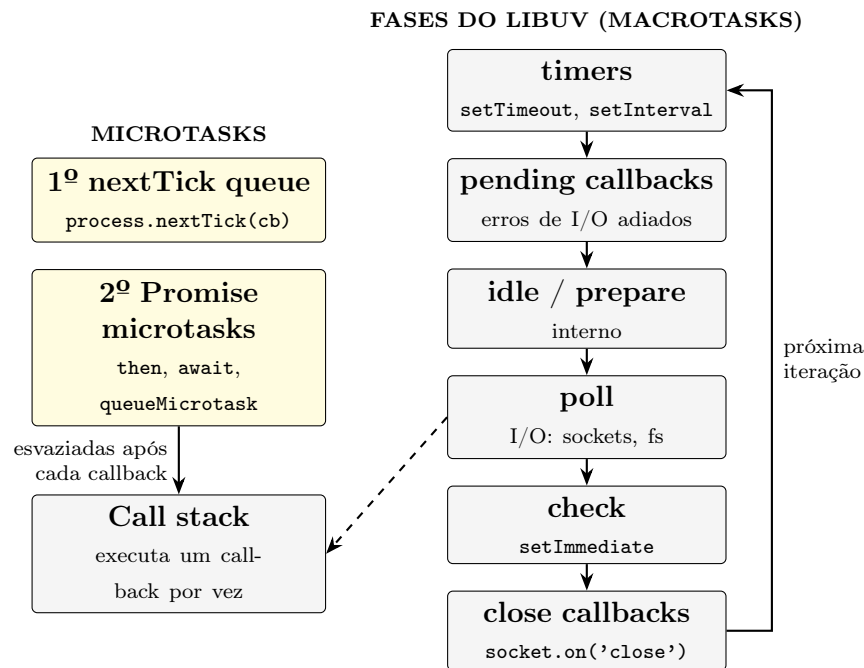


Figure 3.3: Uma iteração do Event Loop no Node.js. Entre cada callback, o Node esvazia a fila de `nextTick` e depois a de Promises. Operações de `fs`, `dns.lookup`, `crypto` e `zlib` usam o thread pool do libuv (4 threads por padrão).

```

1 console.log('A: síncrono');
2 setTimeout(() => console.log('E: timeout'), 0);
3 setImmediate(() => console.log('F: immediate'));
4 Promise.resolve().then(() => console.log('D: promise'));
5 process.nextTick(() => console.log('C: nextTick'));
6 console.log('B: síncrono');
7
8 // CommonJS: A, B, C, D e depois E/F em ordem NÃO garantida no módulo
9   ↳ principal.
10 // Dentro de um callback de I/O, setImmediate sempre roda antes de
11   ↳ setTimeout(0).
12 // ESM: o módulo é avaliado de forma assíncrona, então D costuma sair antes de
13   ↳ C.
  
```

Por que isso importa em produção

Um `JSON.parse` de 50 MB, um loop pesado ou `crypto.pbkdf2Sync` bloqueiam a call stack: nenhuma outra requisição é atendida enquanto isso roda. Um `nextTick` recursivo pode deixar o I/O sem vez (*starvation*).

Domínio: você prevê a ordem de logs mistos, explica a diferença entre micro e macro-tasks e entre `nextTick` e `setImmediate`, e diagnostica “o servidor travou” como bloqueio do Event Loop.

3.12 Prototype

PRIORIDADE ALTA

Todo objeto tem um link interno `[[Prototype]]`. Ao acessar uma propriedade que o objeto não tem, o JS sobe por essa cadeia. `class` é açúcar sintático sobre esse mecanismo: os métodos ficam no `prototype` do construtor, não em cada instância. `prototype` é a propriedade das *funções construtoras*; `__proto__` é o acessor legado do link de *qualquer objeto* — prefira `Object.getPrototypeOf`.

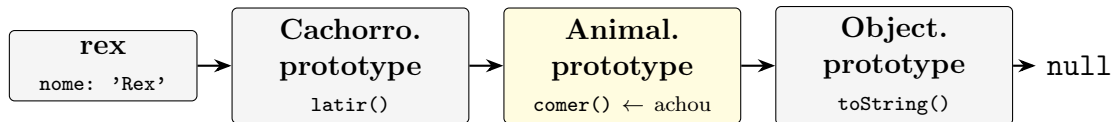


Figure 3.4: Prototype chain. Cada seta é um `[[Prototype]]`. `rex.comer()` não está em `rex` nem em `Cachorro.prototype`; é encontrado em `Animal.prototype`.

```

1 class Animal { comer() { return 'comendo'; } }
2 class Cachorro extends Animal { latir() { return 'au'; } }
3 const rex = new Cachorro();
4
5 Object.getPrototypeOf(rex) === Cachorro.prototype; // true
6 Object.getPrototypeOf(Cachorro.prototype) === Animal.prototype; // true
7 Object.hasOwn(rex, 'latir'); // false -- está no prototype
8 rex instanceof Animal; // true -- percorre a cadeia
9 typeof Cachorro; // 'function'
  
```

Conexão com TypeScript

`instanceof` faz narrowing porque é uma checagem de runtime real. Uma interface não existe em runtime: `x instanceof MinhaInterface` é erro de compilação. Por isso o TS usa tipagem estrutural (Seção 5.1).

Domínio: você desenha a prototype chain de uma hierarquia de classes, sabe onde os métodos realmente vivem e explica por que `instanceof` falha quando existem duas cópias do mesmo pacote em `node_modules`.

3.13 Prioridade média

PRIORIDADE MÉDIA

Você vai usar estes tópicos com frequência, mas eles não bloqueiam o entendimento do TypeScript.

3.13.1 Map, Set, WeakMap e WeakSet

Estrutura	Chaves	Referência	Iterável	Quando usar
Map	qualquer valor	forte	sim	dicionário dinâmico, cache
Set	valores únicos	forte	sim	deduplicação, “já processado?”
WeakMap	só objetos	fraca	não	metadados por objeto sem vazar memória
WeakSet	só objetos	fraca	não	marcar objetos visitados

```

1 const cache = new Map<string, Promise<Usuario>>(); // id -> Promise em
  ↳ andamento
2 const vistos = new Set<string>();
3 const metadados = new WeakMap<object, { criadoEm: number }>(); // some com o
  ↳ objeto
4 const unicos = [...new Set(['a', 'b', 'a'])]; // ['a', 'b']

```

3.13.2 ESM e CommonJS

	ESM	CommonJS
Sintaxe	import / export	require / module.exports
Carregamento	estático, analisado antes de executar; top-level await	dinâmico e síncrono
Exports	live bindings	o valor de module.exports no momento do require
Ativação no Node	.mjs ou "type": "module"	.cjs ou padrão
Imports relativos	extensão obrigatória (./x.js)	extensão opcional
Import dinâmico	await import('./x.js')	require() ou import()

Circular dependencies em CommonJS:

```

1 // a.js
2 exports.pronto = false;
3 const b = require('./b'); // b.js roda AGORA e vê a pela metade
4 exports.pronto = true;
5
6 // b.js
7 const a = require('./a'); // recebe o exports PARCIAL de a
8 console.log(a.pronto); // false
9 // Em ESM, acessar um binding ainda não inicializado lança ReferenceError
  ↳ (TDZ).
10 // Nos dois casos, o ciclo é um problema de design.

```

3.13.3 Outros tópicos de prioridade média

Tópico	O que dominar
Symbol, iterables, iterators	Symbol.iterator é o que faz for...of e spread funcionarem
Intl	formatação de número, moeda e data por locale
JSON	parse com <code>reviver</code> , <code>stringify</code> com <code>replacer</code> ; o que não serializa (Date, BigInt, undefined)
Números	precisão de float, BigInt, Number.isNaN, Number.isFinite
Datas	Date básico, UTC × local, ISO 8601
Regex	o suficiente para validação e parsing simples

```

1 0.1 + 0.2 === 0.3;           // false -- float IEEE 754; dinheiro em
   ↳ centavos inteiros
2 2n ** 64n;                   // 18446744073709551616n (BigInt)
3 Number.isNaN('abc');         // false (o isNaN global faz coerção e retorna
   ↳ true)
4
5 new Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL'
   ↳ }).format(1234.5);
6 // 'R$ 1.234,50'
7
8 JSON.stringify({ id: 1n }, (_, v) => (typeof v === 'bigint' ? v.toString() :
   ↳ v));
9 JSON.parse(texto, (chave, v) => (chave === 'criadoEm' ? new Date(v) : v)); //
   ↳ reviver
10
11 const { default: plugin } = await import(`./plugins/${nome}.js`); // import
   ↳ dinâmico

```

Domínio: você prefere Map a objeto literal para chaves dinâmicas, configura um projeto ESM com TypeScript do zero, entende ERR_REQUIRE_ESM, quebra ciclos de dependência e nunca representa dinheiro com float.

3.14 Prioridade baixa

PRIORIDADE BAIXA

São úteis, mas não devem consumir tempo no início. Saiba que existem e volte quando um problema concreto pedir.

Tópico	O mínimo para agora	Quando voltar
Generators	<code>function*</code> e <code>yield</code> pausam uma função	paginação preguiçosa, pipelines
Proxy e Reflect	interceptam operações em objetos	ORMs, validação, <code>reflect-metadata</code>
Coercion avançada	use <code>===</code> ; conheça <code>truthy/falsy</code>	quase nunca; o TS barra a maioria
Quirks históricos	<code>typeof null === 'object'</code> , ASI	curiosidade, entrevistas
<code>with</code> , <code>eval</code>	existem; não use	nunca, em código de produção
<code>Symbol.toPrimitive</code> , <code>Symbol.hasInstance</code>	customizam coerção e <code>instanceof</code>	metaprogramação em bibliotecas
APIs de browser	DOM, <code>window</code> , <code>localStorage</code>	só se for trabalhar com front-end

Regra de corte

Se o tópico não explica por que um código TypeScript se comporta de certo jeito em runtime e não aparece no seu trabalho atual, ele é prioridade baixa. Anote e siga.

Chapter 4

TypeScript — núcleo principal

A maior parte do seu tempo vai para este capítulo. A progressão vai de *descrever valores* a *derivar tipos de outros tipos*.

Bloco	Você passa a conseguir...	Prioridade
Fundamentos	descrever os dados do domínio	Alta
Classes em TypeScript	modelar serviços, entidades e decorators	Alta
Narrowing	trabalhar com unions com segurança	Alta
Generics	escrever código reutilizável sem perder tipos	Alta
Utility Types	derivar DTOs sem duplicar	Alta
keyof, typeof, indexed access	extrair tipos de valores	Alta
Mapped Types	transformar todas as chaves de um tipo	Alta
Conditional Types	escolher tipos por condição	Alta
infer	extrair partes de tipos	Alta
Template Literal Types	tipar strings estruturadas	Média
satisfies	validar sem perder inferência	Alta
unknown, any, never	lidar com dados externos e exaustividade	Alta
Type Assertions	saber quando (não) mentir ao compilador	Média
Function Overloads	tipar funções com várias formas	Média

4.1 Fundamentos

PRIORIDADE ALTA

Tipos primitivos, arrays e tuples (inclusive `readonly`), enums, union e intersection types, literal types, aliases, interfaces, `as const` e `import type`. Repare na **inferência**: você raramente precisa anotar variáveis locais.

```
1 let id: number = 1;
2 const nome = 'ana';           // tipo literal 'ana' (const não muda)
3 let status = 'ativo';         // string -- let "alarga" o tipo (widening)
4 const ids: readonly number[] = [1, 2];           // sem push/splice
5 const par: readonly [chave: string, valor: number] = ['idade', 30];
```

```

6
7 type Status = 'ativo' | 'inativo' | 'banido';           // union de
  ↳ literais
8 type ComId = { id: string };
9 type ComDatas = { criadoEm: Date; atualizadoEm: Date };
10 type Entidade = ComId & ComDatas;                      // intersection
11
12 interface Usuario {
13   id: string;
14   email: string;
15   apelido?: string;           // opcional
16   readonly criadoEm: Date;    // não pode ser reatribuído (em compile time)
17   status: Status;
18 }
19
20 const ROTAS = { home: '/', pedidos: '/pedidos' } as const; // literais +
  ↳ readonly
21 import type { Pedido } from './pedido.js';              // apagado no emit

```

enum, const enum ou union literal?

```

1 enum Papel { Admin, Leitor }           // gera objeto em runtime; Papel[0] ===
  ↳ 'Admin'
2 const enum Cor { Azul, Verde }         // inlined no emit; problemático com
  ↳ isolatedModules
3
4 const PAPEL = { Admin: 'admin', Leitor: 'leitor' } as const;
5 type Papel2 = (typeof PAPEL)[keyof typeof PAPEL]; // 'admin' | 'leitor' --
  ↳ recomendado

```

Aspecto	interface	type
Objetos	sim	sim
Unions, tuples, primitivos	não	sim
Intersections	via extends	sim (&)
Declaration merging	sim	não
Extensível por terceiros	sim	não
Mapped / conditional	não	sim
Uso recomendado	contratos de objeto	composições e unions

Domínio: você modela um agregado de domínio (Pedido com itens, status e pagamento) sem any, prefere union literal a enum e explica por que `let x = 'a'` vira `string` enquanto `const x = 'a'` vira `'a'`.

4.2 Classes em TypeScript

PRIORIDADE ALTA

- public, private, protected, readonly e parameter properties;
- classes e métodos abstract, override, implements;
- this types (métodos encadeáveis);
- decorators de classe, método e propriedade, e metadata reflection (base do NestJS, TypeORM e class-validator).

```

1 interface Repositorio<T> {
2   buscar(id: string): Promise<T | undefined>;
3 }
4
5 abstract class RepositorioBase<T extends { id: string }> implements
  ↳ Repositorio<T> {
6   protected readonly itens = new Map<string, T>();
7   abstract nomeTabela(): string;
8
9   async buscar(id: string) { return this.itens.get(id); }
10  salvar(item: T): this { // 'this' type: subclasses mantêm o
  ↳ tipo
11    this.itens.set(item.id, item);
12    return this;
13  }
14 }
15
16 class RepositorioPedidos extends RepositorioBase<Pedido> {
17   constructor(private readonly db: Db) { super(); } // parameter property
18   override nomeTabela() { return 'pedidos'; } // override explícito
19 }

```

Decorators no padrão TC39 (TS 5.0+, sem experimentalDecorators):

```

1 function log<This, Args extends unknown[], R>(
2   alvo: (this: This, ...args: Args) => R,
3   contexto: ClassMethodDecoratorContext<This, (this: This, ...args: Args) =>
  ↳ R>,
4 ) {
5   return function (this: This, ...args: Args): R {
6     console.log(`-> ${String(contexto.name)}`, args);
7     return alvo.call(this, ...args);
8   };
9 }
10
11 class ServicoPagamento {
12   @log
13   cobrar(valor: number) { return valor * 1.02; }
14 }

```

Dois sistemas de decorators

NestJS, TypeORM e class-validator usam os decorators *legados* (`experimentalDecorators` + `emitDecoratorMetadata` + `reflect-metadata`), que incluem decorators de *parâmetro* e emitem metadados de tipo em runtime. Os decorators TC39 não têm decorators de parâmetro nem emitem esses metadados. Saiba qual sistema o seu framework usa.

Domínio: você escreve uma hierarquia com `abstract`, `override` e `parameter properties`, implementa um decorator de método simples e explica o que `emitDecoratorMetadata` gera em runtime para a injeção de dependências do NestJS funcionar.

4.3 Narrowing

PRIORIDADE ALTA

Narrowing é o TypeScript **acompanhando checagens de runtime** para estreitar um tipo dentro de um bloco. É onde TS e JS mais se encontram: todo narrowing se apoia em algo que realmente executa.

```

1 function formatar(v: string | number | Date | null): string {
2   if (v === null) return '-'; // equality narrowing
3   if (typeof v === 'string') return v.trim(); // typeof
4   if (v instanceof Date) return v.toISOString(); // instanceof (prototype
    ↪ chain)
5   return v.toFixed(2); // sobrou: number
6 }
7
8 type Peixe = { nadar(): void };
9 type Passaro = { voar(): void };
10 function mover(animal: Peixe | Passaro) {
11   if ('nadar' in animal) animal.nadar(); // in
12   else animal.voar();
13 }
14
15 function tamanho(s?: string) {
16   if (!s) return 0; // truthiness: cuidado, '' também cai aqui
17   return s.length;
18 }

```

Type predicates, assertion functions e discriminated unions:

```

1 type Resultado<T> =
2   | { ok: true; valor: T }
3   | { ok: false; erro: string }; // "ok" é o discriminante
4

```

```

5 function dobrar(r: Resultado<number>) {
6   if (r.ok) return r.valor * 2;      // r: { ok: true; valor: number }
7   return r.erro;                    // r: { ok: false; erro: string }
8 }
9
10 function isString(x: unknown): x is string {      // type predicate
11   return typeof x === 'string';
12 }
13 const nomes = ['a', 1, 'b'].filter(isString);      // string[]
14 // TS 5.5+: .filter(x => typeof x === 'string') já infere o predicate
15
16 function assertDefinido<T>(v: T, msg: string): asserts v is NonNullable<T> {
17   if (v === null || v === undefined) throw new Error(msg); // assertion
18   ↪ function
19 }
19 const url = process.env.DATABASE_URL;
20 assertDefinido(url, 'DATABASE_URL ausente');
21 url.startsWith('postgres');                // url: string daqui em
22 ↪ diante

```

Domínio: você modela estados (pendente | pago | cancelado) como discriminated union, escreve predicates e assertion functions corretos e sabe que um predicate errado engana o compilador.

4.4 Generics

PRIORIDADE ALTA

Generics são **parâmetros de tipo**: permitem que uma função, interface ou classe preserve a relação entre entrada e saída. Constraints (**extends**) limitam o que o parâmetro aceita; defaults (**<T = unknown>**) definem o valor quando nada é informado.

```

1 function primeiro<T>(itens: T[]): T | undefined { // função genérica
2   return itens[0];
3 }
4
5 interface Resposta<T = unknown, E = Error> {      // interface com defaults
6   dados?: T;
7   erro?: E;
8 }
9
10 class Cache<K, V> {                               // classe genérica
11   #mapa = new Map<K, V>();
12   get(chave: K) { return this.#mapa.get(chave); } // V | undefined
13   set(chave: K, valor: V) { this.#mapa.set(chave, valor); return this; }
14 }
15
16 function maiorPor<T extends { length: number }>(a: T, b: T): T { // constraint

```

```

17   return a.length >= b.length ? a : b;
18 }

```

`keyof` e `T[K]` expressam a relação entre chave e valor:

```

1  function atualizarCampo<T, K extends keyof T>(
2      objeto: T,
3      campo: K,
4      valor: T[K]
5  ): T {
6      return {
7          ...objeto,
8          [campo]: valor
9      };
10 }
11
12 const u = { nome: 'Ana', idade: 30 };
13 atualizarCampo(u, 'idade', 31);    // ok
14 atualizarCampo(u, 'idade', '31');  // erro: string não é atribuível a number
15 atualizarCampo(u, 'email', 'x');   // erro: 'email' não é keyof typeof u

```

Heurística

Se um parâmetro de tipo aparece uma única vez na assinatura, ele provavelmente não precisa existir: `function log<T>(x: T): void` é só `function log(x: unknown): void`. Generics servem para **relacionar** tipos.

Domínio: você escreve constraints sem consultar, cria um `Repository<T extends { id: string }>` genérico, explica `K extends keyof T` e reconhece quando um generic é desnecessário.

4.5 Utility Types

PRIORIDADE ALTA

Tipos prontos da biblioteca padrão que transformam outros tipos. Todos são implementados com mapped e conditional types; depois da Seção 4.9 você consegue reescrever cada um.

Utility	O que faz	Uso no backend
<code>Partial<T></code>	todas as propriedades opcionais	body de PATCH
<code>Required<T></code>	todas obrigatórias	config após aplicar defaults
<code>Readonly<T></code>	todas <code>readonly</code> (raso)	value objects
<code>Pick<T, K></code>	mantém só as chaves K	resposta pública
<code>Omit<T, K></code>	remove as chaves K	DTO de criação
<code>Record<K, V></code>	objeto com chaves K e valores V	handler por tipo de evento
<code>Exclude<U, E></code>	remove membros de uma union	<code>Excluído<Status, 'banido'></code>
<code>Extract<U, E></code>	mantém membros atribuíveis a E	subtipo de evento
<code>NonNullable<T></code>	remove <code>null</code> e <code>undefined</code>	após validar presença
<code>ReturnType<F></code>	tipo de retorno de uma função	tipo de uma factory
<code>Parameters<F></code>	tupla dos parâmetros	wrappers e mocks
<code>ConstructorParameters<C></code>	tupla dos parâmetros do construtor	factories de classes
<code>InstanceType<C></code>	tipo da instância de uma classe	containers de DI
<code>Awaited<T></code>	desembrulha <code>Promise</code> (recursivo)	resultado de função <code>async</code>
<code>ThisParameterType<F></code>	tipo do parâmetro <code>this</code>	funções que dependem de contexto
<code>OmitThisParameter<F></code>	remove o parâmetro <code>this</code>	após bind

```

1 interface Usuario {
2   id: string; email: string; senhaHash: string;
3   nome: string; papel: 'admin' | 'leitor'; criadoEm: Date;
4 }
5
6 type CriarUsuarioDTO =
7   Omit<Usuario, 'id' | 'criadoEm' | 'senhaHash'> & { senha: string };
8 type AtualizarUsuarioDTO = Partial<Omit<Usuario, 'id' | 'criadoEm'>>;
9 type UsuarioPublico = Readonly<Pick<Usuario, 'id' | 'nome' | 'papel'>>;
10 type HandlersPorPapel = Record<Usuario['papel'], (u: Usuario) => void>;
11
12 class Conexao { constructor(host: string, porta: number) {} }
13 type ArgsConexao = ConstructorParameters<typeof Conexao>; // [host: string,
14   ↪ porta: number]
15 type Instancia = InstanceType<typeof Conexao>; // Conexao
16
17 function formatar(this: { moeda: string }, v: number) { return `${this.moeda}
18   ↪ ${v}`; }
19 type Contexto = ThisParameterType<typeof formatar>; // { moeda: string }
20 type Solta = OmitThisParameter<typeof formatar>; // (v: number) =>
21   ↪ string

```

Domínio: você deriva DTOs do tipo de domínio em vez de redeclarar campos e sabe a diferença entre `Omit` (chaves de objeto) e `Exclude` (membros de union).

4.6 keyof, typeof e indexed access

PRIORIDADE ALTA

Três ferramentas para **derivar tipos**: `typeof` (em posição de tipo) extrai o tipo de um *valor*; `keyof` extrai a union das chaves de um tipo; `T[K]` acessa o tipo de uma propriedade. Combinadas, fazem do valor a fonte da verdade.

```

1 const config = { porta: 3000, host: 'localhost', debug: false };
2
3 type Config      = typeof config;           // { porta: number; host: string; ...
   ↪ }
4 type ChaveConfig = keyof Config;            // 'porta' | 'host' | 'debug'
5 type Porta      = Config['porta'];          // number
6 type Valores    = Config[keyof Config];     // number | string | boolean
7
8 const EVENTOS = ['criado', 'pago', 'cancelado'] as const;
9 type Evento = (typeof EVENTOS)[number];    // 'criado' | 'pago' | 'cancelado'
10
11 async function buscarUsuario(id: string) { return { id, nome: 'Ana' }; }
12 type UsuarioDb = Awaited<ReturnType<typeof buscarUsuario>>;

```

Expressão	Onde existe	Resultado
<code>typeof x === 'string'</code>	runtime (JS)	uma string: <code>'string'</code> , <code>'object'</code> ...
<code>type T = typeof x</code>	compile time (TS)	o tipo estático de <code>x</code>
<code>keyof T</code>	compile time	union das chaves
<code>Object.keys(x)</code>	runtime	<code>string[]</code>

Domínio: você cria uma union a partir de um array `as const` e explica por que `Object.keys` retorna `string[]`, e não `(keyof T)[]`: a tipagem estrutural permite propriedades extras.

4.7 Mapped Types

PRIORIDADE ALTA

Um mapped type percorre as chaves de um tipo e produz um novo tipo — um `for...in` em nível de tipo. Modificadores `+`, `-`, `+readonly` e `-readonly` adicionam ou removem opcionalidade e imutabilidade; a cláusula `as` renomeia ou filtra chaves.

```

1 type Opcional<T> = {
2   [K in keyof T]?: T[K];
3 };
4

```

```

5 type SomenteLeitura<T> = { +readonly [K in keyof T]: T[K] };
6 type Mutavel<T>         = { -readonly [K in keyof T]: T[K] }; // remove
  ↪ modificador
7 type Obrigatorio<T>     = { [K in keyof T]-?: T[K] };           // remove o '?'
8
9 type Validadores<T> = { [K in keyof T]: (valor: T[K]) => boolean };
10 const validarUsuario: Validadores<{ email: string; idade: number }> = {
11   email: (v) => v.includes('@'), // v: string
12   idade: (v) => v >= 18,         // v: number
13 };
14
15 type SemFuncoes<T> = { [K in keyof T as T[K] extends Function ? never : K]:
  ↪ T[K] };

```

Domínio: você reescreve Partial, Readonly e Record de memória, usa key remapping com as e cria tipos como Validadores<T> para o seu domínio.

4.8 Conditional Types

PRIORIDADE ALTA

A forma `T extends U ? X : Y` significa: se `T` é atribuível a `U`, o resultado é `X`; senão, `Y`. Quando `T` é um parâmetro “nu” e recebe uma union, a condição é aplicada a **cada membro** (distributividade). Para evitar, envolva em tupla: `[T] extends [U]`.

```

1 type EhString<T> = T extends string ? true : false;
2 type A = EhString<'x'>; // true
3 type B = EhString<42>; // false
4
5 type ParaArray<T> = T extends unknown ? T[] : never;
6 type C = ParaArray<string | number>; // string[] | number[] (distribui)
7
8 type ParaArrayJunto<T> = [T] extends [unknown] ? T[] : never;
9 type D = ParaArrayJunto<string | number>; // (string | number)[] (não
  ↪ distribui)
10
11 type MeuExclui<T, U> = T extends U ? never : T; // é assim que Exclui
  ↪ funciona
12 type E = MeuExclui<'a' | 'b' | 'c', 'a'>; // 'b' | 'c'

```

Domínio: você explica por que Exclui funciona (distributividade + never desapeça da union) e sabe desligar a distribuição com `[T]`.

4.9 infer

PRIORIDADE ALTA

`infer` declara uma variável de tipo dentro de um conditional type, capturando uma parte da estrutura. Do simples ao avançado, incluindo conditional types aninhados e variadic tuples:

```

1 // nível 1 -- elemento de um array
2 type Elemento<T> = T extends (infer E)[] ? E : never;
3 type N1 = Elemento<string>; // string
4
5 // nível 2 -- reimplementando ReturnType
6 type Retorno<F> = F extends (...args: any[]) => infer R ? R : never;
7 type N2 = Retorno<() => Promise<number>>; // Promise<number>
8
9 // nível 3 -- desembrulhar Promise recursivamente (ideia do Awaited)
10 type Desembrulhar<T> = T extends Promise<infer V> ? Desembrulhar<V> : T;
11 type N3 = Desembrulhar<Promise<Promise<number>>>; // number
12
13 // nível 4 -- cabeça e cauda de tupla (variadic)
14 type Cabeça<T extends unknown[]> = T extends [infer H, ...unknown[]] ? H :
   ↳ never;
15 type Cauda<T extends unknown[]> = T extends [unknown, ...infer R] ? R : [];
16 type N4 = Cauda<[string, number, boolean]>; // [number, boolean]
17
18 // nível 5 -- infer em template literal: parâmetros de rota
19 type ParamsRota<S extends string> =
20   S extends `${string}:${infer P}/${infer Resto}`
21     ? P | ParamsRota<`${Resto}`>
22     : S extends `${string}:${infer P}` ? P : never;
23
24 type N5 = ParamsRota<'/usuarios/:id/pedidos/:pedidoId'>; // 'id' | 'pedidoId'

```

Domínio: você implementa versões simplificadas de `ReturnType`, `Parameters` e `Awaited` e lê o tipo do nível 5 explicando cada passo da recursão.

4.10 Template Literal Types

PRIORIDADE MÉDIA

A mesma sintaxe das template strings, em nível de tipo. Combinadas com unions, geram todas as combinações; com mapped types (cláusula `as`), renomeiam chaves. Os intrínsecos `Uppercase`, `Lowercase`, `Capitalize` e `Uncapitalize` transformam literais.

```

1 type Metodo = 'GET' | 'POST' | 'DELETE';
2 type Recurso = 'usuarios' | 'pedidos';

```

```

3 type Permissao = `${Lowercase<Metodo>}:${Recurso}`;
4 // 'get:usuarios' / 'get:pedidos' / 'post:usuarios' / ... (6 combinações)
5
6 type Eventos<T> = {
7   [K in keyof T & string as `on${Capitalize<K>}Alterado`]: (novo: T[K]) =>
8     → void;
9 }
10 // Path<T>: todos os caminhos com ponto de um objeto aninhado
11 type Path<T> = T extends object
12   ? { [K in keyof T & string]: T[K] extends object ? K | `${K}.${Path<T[K]>}` :
13     → K }[keyof T & string]
14   : never;
15
16 type Config = { db: { host: string; porta: number }; debug: boolean };
17 type Caminhos = Path<Config>; // 'db' / 'db.host' / 'db.porta' / 'debug'

```

Domínio: você tipa chaves de cache, nomes de eventos, permissões ou caminhos de configuração com template literal types e usa Capitalize/Uppercase em key remapping.

4.11 satisfies

PRIORIDADE ALTA

Uma **anotação** (`const x: T = ...`) substitui o tipo inferido por `T` e alarga os literais. O operador `satisfies` (TS 4.9+) **verifica** se o valor é compatível com `T`, mas mantém o tipo inferido, mais preciso.

```

1 // anotação: tipo declarado, literais alargados
2 const config: Record<string, string> = { host: 'localhost' };
3 config.host; // string
4 config.naoExiste; // compila: qualquer string é chave válida
5
6 // satisfies: valida a forma e preserva o tipo inferido
7 const config2 = { host: 'localhost' } satisfies Record<string, string>;
8 config2.host; // string (chave conhecida)
9 config2.naoExiste; // erro: a chave não existe
10
11 const config3 = { host: 'localhost' } as const satisfies Record<string,
12   → string>;
13 config3.host; // 'localhost' -- o literal, readonly
14
15 type Cor = string | [number, number, number];
16 const paleta = { primaria: [49, 120, 198], fundo: '#0b0f17' } satisfies
17   → Record<string, Cor>;
18 paleta.fundo.toUpperCase(); // ok: o TS lembra que é string, não Cor
19 paleta.primaria[0]; // number -- tupla preservada

```

	Anotação : T	satisfies T	as T
Verifica compatibilidade	sim	sim	quase nada
Tipo resultante	T (alargado)	o inferido	T
Checa propriedades extras	sim	sim	não
Uso ideal	contratos, parâmetros	configs, mapas, rotas	raramente

Domínio: você usa `as` `const` `satisfies` em tabelas de configuração e rotas e explica em uma frase quando preferir `satisfies`, `anotação` ou `as` `const`.

4.12 unknown, any e never

PRIORIDADE ALTA

Tipo	Significa	Pode usar direto?	Quando usar
any	desliga a checagem	sim — e contamina o resto	quase nunca; migração de JS
unknown	o “any seguro”	não: precisa de narrowing	dados externos: JSON, body, catch
never	“não pode existir”	—	exhaustive checks, funções que sempre lançam

```

1 function lerJson(texto: string): unknown {
2   return JSON.parse(texto);           // JSON.parse retorna any; "fechamos" em
   ↳ unknown
3 }
4 const dado = lerJson(body);
5 dado.nome;                           // erro: 'dado' is of type 'unknown'
6 if (typeof dado === 'object' && dado !== null && 'nome' in dado) {
7   dado.nome;                          // ok, depois de provar
8 }
9
10 try { await pagar(); } catch (err) { // err: unknown (strict)
11   const msg = err instanceof Error ? err.message : String(err);
12 }

```

Exhaustive check com `never`:

```

1 function assertNever(x: never): never {
2   throw new Error(`Caso não tratado: ${JSON.stringify(x)}`);
3 }
4
5 type Pagamento =
6   | { tipo: 'pix'; chave: string }
7   | { tipo: 'cartao'; bandeira: string }

```

```

8 | { tipo: 'boleto'; vencimento: Date };
9
10 function taxa(p: Pagamento): number {
11   switch (p.tipo) {
12     case 'pix':      return 0;
13     case 'cartao':   return 0.03;
14     case 'boleto':   return 2.5;
15     default:         return assertNever(p); // novo tipo na union? erro AQUI
16   }
17 }

```

Domínio: seu código de fronteira (HTTP, filas, variáveis de ambiente) recebe `unknown` e valida; você usa `assertNever` para que um caso novo quebre a compilação em todo `switch` esquecido.

4.13 Type Assertions

PRIORIDADE MÉDIA

`as` e a non-null assertion `!` não convertem nem checam nada em runtime: eles **mandam o compilador acreditar em você**.

```

1 const usuario = corpo as Usuario;           // nenhuma validação acontece
2 const porta = process.env.PORT!;             // "não é undefined" -- e se for?
3 const n = 'texto' as unknown as number;      // double assertion: alarme vermelho
4 const rotas = ['/a', '/b'] as const;         // as const: outra coisa, congela
   ↪ literais
5
6 // melhor: prove em runtime e deixe o narrowing trabalhar
7 const portaSegura = Number(process.env.PORT ?? 3000);

```

Quando é aceitável: depois de uma validação que o compilador não consegue acompanhar; em testes (mocks parciais); ao contornar uma biblioteca mal tipada. Para configurações, `satisfies` é a alternativa segura.

Riscos: esconde bugs até a produção e sobrevive a refatorações sem avisar. Usar `!` em `Map.get`, `find` e variáveis de ambiente é a origem clássica de `Cannot read properties of undefined`.

Domínio: você justifica cada `as` e `!` do seu código e troca a maioria por `narrowing`, `satisfies` ou validação de schema.

4.14 Function Overloads

PRIORIDADE MÉDIA

Overloads declaram várias assinaturas para uma única implementação. Use quando o **tipo de retorno depende do tipo do argumento** de um jeito que uma union não expressa.

```
1 function buscar(id: string): Promise<Usuario>;
2 function buscar(ids: string[]): Promise<Usuario[]>;
3 function buscar(arg: string | string[]): Promise<Usuario | Usuario[]> {
4     return Array.isArray(arg) ? repo.buscarVarios(arg) : repo.buscarUm(arg);
5 }
6 const um      = await buscar('u1');           // Usuario
7 const varios  = await buscar(['u1', 'u2']);    // Usuario[]
8
9 // quando o retorno NÃO depende da entrada, uma union basta:
10 function registrar(erro: string | Error): void { /* ... */ }
```

Cuidado

A assinatura de implementação não é visível para quem chama, e o TS não verifica se a implementação respeita cada overload. Se der para resolver com union ou com generic + conditional type, prefira.

Domínio: você decide entre union, generic e overload olhando apenas para a relação entrada → saída.

Chapter 5

TypeScript avançado

O que separa quem usa TypeScript de quem projeta com TypeScript.

Tópico	Resumo	Prioridade
Structural typing	compatibilidade pela forma, não pelo nome	Alta
Configuração do <code>tsconfig</code>	as flags definem quão seguro é o projeto	Alta
Variance	co-, contra-, bi- e invariância	Média
Tipos recursivos	JSON, árvores, <code>DeepPartial</code>	Média
Branded / opaque types	IDs que não se misturam	Média
Variadic tuple types	manipular listas de argumentos	Média
Declaration files <code>.d.ts</code>	próprios, <code>@types/*</code> , ambientes e globais	Média
Module augmentation	estender tipos de bibliotecas	Média
Tipos de bibliotecas externas	<code>@types</code> , <code>exports</code> , <code>skipLibCheck</code>	Média
Conditional e mapped complexos	filtrar chaves, gerar APIs tipadas	Baixa
Declaration merging e <code>namespace</code>	fusão de declarações; <code>namespace</code> é legado	Baixa
Inferência avançada	<code>const</code> type parameters, <code>NoInfer</code>	Baixa
Type-level programming	recursão com acumuladores	Baixa

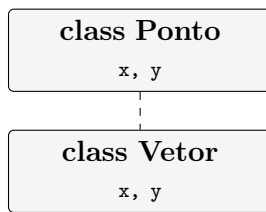
5.1 Structural Typing

PRIORIDADE ALTA

Um tipo é compatível com outro se tiver **a forma exigida**; o nome da classe ou interface é irrelevante. Isso espelha o *duck typing* do JavaScript, onde interfaces não existem em runtime. Linguagens como Java e C# usam *nominal typing*: dois tipos com a mesma forma, mas nomes diferentes, são incompatíveis.

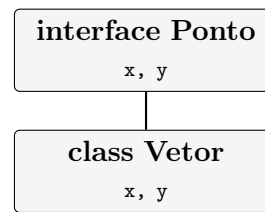
```
1 interface Ponto { x: number; y: number }
2 class Pixel { constructor(public x: number, public y: number, public cor:
  ↳ string) {} }
3
4 const p: Ponto = new Pixel(1, 2, 'red'); // ok: tem x e y
5 const q: Ponto = { x: 1, y: 2, cor: 'red' }; // erro: excess property check
```

Nominal (Java, C#)



× incompatíveis
nomes diferentes

Structural (TypeScript)



✓ compatíveis
mesma forma

Figure 5.1: Nominal typing compara nomes; structural typing compara formas.

```
6 const tmp = { x: 1, y: 2, cor: 'red' };
7 const r: Ponto = tmp;           // ok: a checagem extra só vale
  ↪ para literais
```

5.2 Variance

PRIORIDADE MÉDIA

Variância	Significa	Onde aparece
Covariância	F<Sub> é atribuível a F<Super>	retornos, arrays readonly
Contravariância	F<Super> é atribuível a F<Sub>	parâmetros de função
Bivariância	as duas direções são aceitas	parâmetros de <i>métodos</i> (unsound)
Invariância	nenhuma direção	tipos lidos e escritos (in out T)

```
1 class Animal { comer() {} }
2 class Cachorro extends Animal { latir() {} }
3
4 let animais: Animal[] = [];
5 let caes: Cachorro[] = [];
6 animais = caes;           // ok: arrays são covariantes (unsound: animais.push(new
  ↪ Animal()))
7
8 type Handler<T> = (valor: T) => void;
9 let hAnimal: Handler<Animal> = (a) => a.comer();
10 let hCao: Handler<Cachorro> = (c) => c.latir();
11 hCao = hAnimal;           // ok: parâmetros são contravariantes
12 hAnimal = hCao;           // erro com strictFunctionTypes: chamaria latir() num
  ↪ Animal
13
14 interface Caixa<in out T> { valor: T } // variance annotation explícita (TS
  ↪ 4.7+)
```

Métodos declarados com sintaxe de método (`m(x: T): void`) continuam bivariantes mesmo com `strictFunctionTypes`; propriedades do tipo função (`m: (x: T) => void`) são checadas.

5.3 Tipos recursivos, variadic tuples e type-level programming

PRIORIDADE MÉDIA

```

1 type Json = string | number | boolean | null | Json[] | { [chave: string]: Json
  ↪   };
2
3 type DeepPartial<T> = {
4   [K in keyof T]?: T[K] extends object ? DeepPartial<T[K]> : T[K];
5 };
6
7 // chaves cujo valor é de um certo tipo
8 type ChavesDoTipo<T, V> = { [K in keyof T]: T[K] extends V ? K : never }[keyof
  ↪   T];
9
10 // variadic tuple types
11 type Concat<A extends unknown[], B extends unknown[]> = [...A, ...B];
12 type R = Concat<string[], [number, boolean]>; // [string, number, boolean]
13
14 function comContexto<A extends unknown[], T>(fn: (ctx: Contexto, ...args: A) =>
  ↪   T) {
15   return (...args: A) => fn(contextoAtual(), ...args); // remove o 1º
  ↪   parâmetro
16 }
17
18 // type-level programming: recursão com acumulador
19 type Tupla<N extends number, Acc extends unknown[] = []> =
20   Acc['length'] extends N ? Acc : Tupla<N, [...Acc, unknown]>;
21 type Soma<A extends number, B extends number> = [...Tupla<A>,
  ↪   ...Tupla<B>]['length'];
22 type Cinco = Soma<2, 3>; // 5

```

Use com moderação

Tipos complexos têm custo: mensagens de erro ilegíveis, compilação mais lenta e colegas que não conseguem manter o código. Programação em nível de tipo compensa em bibliotecas; em código de aplicação, prefira tipos simples e explícitos.

5.4 Branded Types

PRIORIDADE MÉDIA

Como a tipagem é estrutural, `UsuarioId` e `PedidoId` declarados como `string` se misturam. Um *brand* (ou *opaque type*) adiciona uma propriedade fantasma que só existe em compile time, simulando tipagem nominal.

```

1 type Brand<T, B extends string> = T & { readonly __brand: B };
2 type UsuarioId = Brand<string, 'UsuarioId'>;
3 type PedidoId = Brand<string, 'PedidoId'>;
4
5 function paraUsuarioId(v: string): UsuarioId { // "smart constructor": valide
  ↪ aqui
6   if (!v.startsWith('usr_')) throw new Error('id inválido');
7   return v as UsuarioId; // a única assertion, num só
  ↪ lugar
8 }
9
10 declare function buscarPedido(id: PedidoId): Promise<unknown>;
11 const uid = paraUsuarioId('usr_42');
12 buscarPedido(uid); // erro: UsuarioId não é PedidoId -- mesmo sendo string em
  ↪ runtime

```

5.5 Declaration files, module augmentation e declaration merging

PRIORIDADE MÉDIA

Três conceitos que costumam ser confundidos:

- **Declaration file (.d.ts):** só declarações, sem implementação. Pode ser seu, vir de `@types/*`, declarar um *módulo ambiente* (`declare module 'x'`) ou globais ambientes (`declare global`).
- **Declaration merging:** duas declarações com o mesmo nome se fundem (duas interface `Foo`, ou namespace + função).
- **Module augmentation:** usar declaration merging para *estender um módulo existente* a partir de outro arquivo.

```

1 // types/lib-legada.d.ts -- módulo ambiente para uma lib JS sem tipos
2 declare module 'lib-legada' {
3   export function calcular(valor: number, taxa?: number): number;
4   export const versao: string;
5 }

```

```

1 // env.d.ts -- global ambiente
2 declare global {
3   namespace NodeJS {
4     interface ProcessEnv {
5       DATABASE_URL: string;
6       NODE_ENV: 'development' | 'production' | 'test';

```

```
7     }  
8   }  
9 }  
10 export {};
```

```
1 // fastify.d.ts -- module augmentation  
2 import 'fastify';  
3  
4 declare module 'fastify' {  
5   interface FastifyRequest {           // fundida com a interface original  
6     usuario?: { id: string; papel: 'admin' | 'leitor' };  
7   }  
8 }
```

namespace é o sistema de módulos anterior ao ESM. Ainda aparece em `.d.ts` (como NodeJS acima) e em código legado, mas não o use em código novo: use módulos.

Lembre-se do runtime

Augmentation e `.d.ts` apenas *declaram*. Tipar `DATABASE_URL: string` não garante que a variável exista: valide o ambiente na inicialização (Seção 6.4) e só então confie no tipo.

5.6 Tipos de bibliotecas externas

PRIORIDADE MÉDIA

- Bibliotecas escritas em TS publicam seus `.d.ts`; as demais usam `@types/nome` (DefinitelyTyped). `@types/node` deve acompanhar a versão do Node que você usa.
- O campo `exports` do `package.json` (com a condição `types`) decide o que o TS enxerga; por isso `moduleResolution` precisa refletir o runtime.
- Cuidado com `any` escondido em tipos de bibliotecas: ele se espalha pelo seu código sem aviso.
- Aprenda a ler um `.d.ts` (“Go to Definition” no editor): é a documentação mais precisa de qualquer biblioteca.

5.7 Inferência avançada

PRIORIDADE BAIXA

```
1 function rotas<const T extends readonly string[]>(r: T) { return r; } // TS
   ↪ 5.0+
2 const r = rotas(['/a', '/b']); // readonly ['/a', '/b'] sem 'as const'
3
4 function criarMaquina<S extends string>(estados: S[], inicial: NoInfer<S>) {}
   ↪ // TS 5.4+
5 criarMaquina(['aberto', 'fechado'], 'pendente'); // erro: 'inicial' não infere
   ↪ S
```

5.8 Configuração do tsconfig

PRIORIDADE ALTA

Um ponto de partida sólido para backend Node.js moderno com ESM:

```
1 {
2   "compilerOptions": {
3     "target": "ES2023",
4     "lib": ["ES2023"],
5     "module": "NodeNext",
6     "moduleResolution": "NodeNext",
7     "strict": true,
8     "noUncheckedIndexedAccess": true,
9     "exactOptionalPropertyTypes": true,
10    "noImplicitOverride": true,
11    "verbatimModuleSyntax": true,
12    "isolatedModules": true,
13    "skipLibCheck": true,
14    "incremental": true,
15    "sourceMap": true,
16    "outDir": "dist",
17    "rootDir": "src"
18  },
19  "include": ["src"]
20 }
```

Flag	O que faz
<code>strict</code>	liga a família de checagens estritas: <code>strictNullChecks</code> , <code>noImplicitAny</code> , <code>strictFunctionTypes</code> , <code>useUnknownInCatchVariables...</code>
<code>exactOptionalPropertyTypes</code>	diferencia <code>undefined</code> explícito de propriedade ausente
<code>noUncheckedIndexedAccess</code>	<code>arr[i]</code> e <code>obj[chave]</code> passam a ser <code>T undefined</code>
<code>noImplicitOverride</code>	exige <code>override</code> explícito em subclasses
<code>isolatedModules</code>	garante compatibilidade com transpilers (esbuild, swc, tsx)
<code>verbatimModuleSyntax</code>	força <code>import type</code> explícito; o emit fica previsível
<code>module</code>	<code>commonjs</code> , <code>esnext</code> , <code>nodenext</code> : formato de módulo emitido
<code>moduleResolution</code>	<code>node</code> (legado), <code>bundler</code> , <code>node16</code> , <code>nodenext</code> : como imports são resolvidos
<code>target</code>	versão de JavaScript emitida
<code>lib</code>	bibliotecas de tipos disponíveis (ex.: sem DOM no backend)
<code>paths / baseUrl</code>	alias de import — <i>só em compile time</i>
<code>skipLibCheck</code>	pula a checagem de <code>.d.ts</code> de terceiros
<code>incremental / tsBuildInfoFile</code>	builds incrementais
<code>composite + project references</code>	monorepos com build por pacote
<code>erasableSyntaxOnly</code>	proíbe sintaxe que gera código, para rodar <code>.ts</code> direto no Node

paths não existe em runtime

`paths` faz o TypeScript *entender* `import x from '@servicos'`, mas o `tsc` não reescreve esse import no JS emitido: o Node falha com `ERR_MODULE_NOT_FOUND`. Use um bundler, `tsc-alias`, ou os *subpath imports* nativos do Node (`"imports": { "#src/*": "./dist/*" }` no `package.json`).

NestJS e outros frameworks com decorators legados exigem `experimentalDecorators` e `emitDecoratorMetadata` e costumam usar `module: commonjs`. Referências prontas: o repositório `tsconfig/bases`.

Domínio: você configura um projeto strict do zero, explica cada flag do seu `tsconfig`, tipa uma biblioteca sem tipos com um `.d.ts`, estende o request do seu framework via `augmentation` e usa `branded types` para IDs.

Chapter 6

Ferramentas e ecossistema TypeScript

Uma stack moderna em TypeScript não se resume à linguagem. Estas ferramentas são parte do domínio.

6.1 Execução e build

Ferramenta	O que faz	Checa tipos?
<code>tsc</code>	compilador oficial: checa e emite JS e <code>.d.ts</code>	sim (o único)
<code>tsx</code>	executa <code>.ts</code> em desenvolvimento (via <code>esbuild</code>)	não
<code>ts-node</code>	executa <code>.ts</code> ; pode checar tipos (lento)	opcional
<code>swc</code> / <code>esbuild</code>	transpiladores muito rápidos	não
<code>tsup</code>	bundler para bibliotecas, gera <code>.d.ts</code>	via <code>tsc</code>
Node (type stripping)	executa <code>.ts</code> apagando tipos	não
Bun	runtime alternativo com TS nativo	não

Consequência prática

Quase toda ferramenta rápida *só apaga tipos*. Rode `tsc -noEmit` no CI (e no editor): é ele quem garante que o código está correto em nível de tipos.

6.2 Lint e formatação

- **ESLint** + `typescript-eslint`: habilite as regras *type-aware*. `no-floating-promises` e `no-misused-promises` pegam promises esquecidas, uma das maiores fontes de bug em backend.
- **Prettier**: formatação, sem discussão de estilo em code review.

- **Biome**: alternativa integrada (lint + format) e muito mais rápida.

6.3 Testes e type tests

- **Vitest** (rápido, ESM nativo) ou **Jest** + **ts-jest**.
- Mocks tipados: `vi.fn<typeof fn>()` ou `jest.Mocked<T>`.
- **Type tests** com **tsd**, **expect-type** ou o `expectTypeOf` do Vitest: testam os *tipos*, não o runtime.

```
1 import { expectTypeOf, test } from 'vitest';
2
3 test('atualizarCampo preserva o tipo', () => {
4   expectTypeOf(atualizarCampo({ a: 1 }, 'a', 2)).toEqualTypeOf<{ a: number
5     ↳ }>();
6   // @ts-expect-error -- 'b' não é chave do objeto
7   atualizarCampo({ a: 1 }, 'b', 2);
8 });
```

6.4 Validação em runtime (essencial)

O TypeScript *não* valida em runtime — os tipos já não existem quando a requisição chega. Todo dado externo (HTTP, filas, variáveis de ambiente, banco, APIs de terceiros) precisa ser validado. Bibliotecas de schema resolvem os dois lados de uma vez: validam em runtime e *inferem* o tipo estático.

```
1 import { z } from 'zod';
2
3 const CriarPedido = z.object({
4   clienteId: z.string().min(1),
5   itens: z.array(z.object({ sku: z.string(), qtd: z.number().int().positive()
6     ↳ }))).min(1),
7 });
8
9 type CriarPedido = z.infer<typeof CriarPedido>; // tipo derivado do schema
10
11 const resultado = CriarPedido.safeParse(req.body); // runtime
12 if (!resultado.success) return res.status(400).send(resultado.error.issues);
13 const pedido: CriarPedido = resultado.data; // compile time + runtime
14 // alinhados
15
16 // variáveis de ambiente tipadas: falha no boot, não no meio de uma requisição
17 const Env = z.object({
18   NODE_ENV: z.enum(['development', 'production', 'test']),
19   PORT: z.coerce.number().default(3000),
20   DATABASE_URL: z.string().url(),
21 });
```

```

18 });
19 export const env = Env.parse(process.env);

```

Biblioteca	Característica
Zod	o padrão de fato; API expressiva, ecossistema grande
Valibot	modular e pequeno (tree-shaking)
ArkType	sintaxe próxima de tipos TS, muito rápido
TypeBox	gera JSON Schema; integração natural com Fastify
io-ts	funcional, legado

6.5 Frameworks, banco de dados e configuração

Ferramenta	Como lida com tipos
NestJS	decorators legados, DI por metadados, módulos
Fastify	schemas (JSON Schema/TypeBox) tipam request e response
Express	tipos via <code>@types/express</code> ; genéricos em <code>Request<P, ResBody, ReqBody></code>
Hono	leve, multi-runtime, inferência forte nas rotas
tRPC	tipos compartilhados cliente-servidor sem geração de código
Prisma	schema próprio → client gerado e totalmente tipado
Drizzle	schema escrito em TS; queries com aparência de SQL
TypeORM	entidades como classes com decorators; tipagem mais frouxa
Kysely	query builder type-safe; tipos do banco via codegen

Migrations devem ser versionadas e, sempre que possível, geradas a partir do mesmo schema que tipa o código (Prisma Migrate, Drizzle Kit). Configurações da aplicação: objetos com `as const satisfies` (Seção 4.11) e variáveis de ambiente validadas com Zod.

Domínio: você configura ESLint + Prettier + TS em um projeto novo, escreve testes (inclusive de tipos) com Vitest ou Jest, valida input com Zod, configura um ORM tipado e faz build com `tsup` ou `esbuild` sabendo que só o `tsc` checa tipos.

Chapter 7

TypeScript + JavaScript na prática

Em código real, os dois nunca aparecem separados. Em cada exemplo, observe o que é responsabilidade do compilador e o que é responsabilidade do runtime.

7.1 Closure + Generic

```
1 function criarRepositorio<T>() {
2   const dados: T[] = [];
3
4   return {
5     adicionar(item: T) {
6       dados.push(item);
7     },
8
9     listar() {
10      return dados;
11    }
12  };
13 }
14
15 const usuarios = criarRepositorio<{ id: string; nome: string }>();
16 usuarios.adicionar({ id: '1', nome: 'Ana' });
17 usuarios.adicionar({ id: 2 }); // erro de compilação
18 usuarios.listar()[0].nome; // string
```

- **O TypeScript controla os tipos:** item e o retorno de listar() são tipados por instância.
- **A closure mantém o estado:** o array dados fica privado e vivo entre chamadas.
- **O JavaScript executa a lógica em runtime:** o push acontece de fato, e listar() devolve a *mesma referência* do array — quem chama pode mutá-lo.

7.2 Promise + Generics

```

1 function comTimeout<T>(promessa: Promise<T>, ms: number): Promise<T> {
2   let timer: NodeJS.Timeout;
3   const limite = new Promise<never>((_, rejeitar) => {
4     timer = setTimeout(() => rejeitar(new Error(`timeout após ${ms}ms`)), ms);
5   });
6   return Promise.race([promessa, limite]).finally(() => clearTimeout(timer));
7 }
8
9 async function comRetry<T>(fn: () => Promise<T>, tentativas = 3): Promise<T> {
10  try {
11    return await fn(); // 'return await' para o catch funcionar
12  } catch (err) {
13    if (tentativas <= 1) throw err;
14    return comRetry(fn, tentativas - 1);
15  }
16 }
17
18 const usuario = await comTimeout(comRetry(() => buscarUsuario('1')), 2000);

```

- **TypeScript:** T atravessa as duas funções; Promise<never> não “suja” o tipo do race.
- **Closure:** guarda timer e tentativas entre callbacks.
- **JavaScript:** executa a corrida, o timer e o finally; a promise perdedora *não* é cancelada (use AbortController para cancelar de verdade).

7.3 Type guard + runtime check

```

1 interface CriarPedido { clienteId: string; itens: { sku: string; qtd: number
  ↪ }[] }
2
3 function isCriarPedido(x: unknown): x is CriarPedido {
4   if (typeof x !== 'object' || x === null) return false;
5   const o = x as Record<string, unknown>;
6   return typeof o.clienteId === 'string'
7     && Array.isArray(o.itens)
8     && o.itens.every((i) => typeof i?.sku === 'string' &&
  ↪   Number.isInteger(i?.qtd));
9 }
10
11 app.post('/pedidos', (req, res) => {
12   const body: unknown = req.body;
13   if (!isCriarPedido(body)) return res.status(400).send({ erro: 'payload
  ↪   inválido' });
14   body.itens.map((i) => i.sku); // body: CriarPedido daqui em diante

```

```
15 });
```

- **TypeScript:** faz o narrowing de unknown para CriarPedido depois do if.
- **Contrato:** o predicate é uma promessa — se a função estiver errada, o TS acredita mesmo assim.
- **JavaScript:** executa as checagens reais (typeof, Array.isArray). Em produção, prefira um schema (Seção 6.4), que é a mesma ideia com uma única fonte da verdade.

7.4 Classes + Prototype

```
1 class Conta {
2   #saldo = 0; // privado em RUNTIME (JavaScript)
3   private historico: string[] = []; // privado só em COMPILE TIME
4   ↪ (TypeScript)
5   constructor(readonly titular: string) {}
6
7   depositar(valor: number) {
8     this.#saldo += valor;
9     this.historico.push(`+${valor}`);
10    return this;
11  }
12  get saldo() { return this.#saldo; }
13 }
14
15 const c = new Conta('ana');
16 Object.getPrototypeOf(c) === Conta.prototype; // true -- depositar vive no
17 ↪ prototype // funciona: 'private' foi
18 (c as any).historico; // apagado
19 setTimeout(c.depositar, 0, 10); // compila! e quebra: this é
20 ↪ undefined
```

- **TypeScript:** controla private, readonly e a parameter property titular (sintaxe que gera código).
- **Prototype:** métodos são compartilhados via Conta.prototype; campos ficam na instância.
- **JavaScript:** #saldo é realmente inacessível de fora; this segue as regras do call site.

7.5 async/await + Promise

```
1 async function carregarPainel(id: string) {
```

```

2   const [usuario, pedidos, saldo] = await Promise.all([
3       buscarUsuario(id),      // Promise<Usuario>
4       buscarPedidos(id),      // Promise<Pedido[]>
5       calcularSaldo(id),      // Promise<number>
6   ]);                          // TS infere a tupla [Usuario, Pedido[], number]
7   return { usuario, pedidos, saldo };
8 }
9
10 type Painel = Awaited<ReturnType<typeof carregarPainel>>;
11
12 const resultados = await Promise.allSettled(emails.map(enviarEmail));
13 const falhas = resultados.filter(
14     (r): r is PromiseRejectedResult => r.status === 'rejected'
15 );

```

- **TypeScript:** infere a tupla do `Promise.all` e faz o narrowing do `allSettled`.
- **Event Loop:** as três buscas estão em voo ao mesmo tempo; o `await` libera a thread.
- **JavaScript:** a primeira rejeição encerra o `all`, mas as outras operações continuam rodando.

7.6 EventEmitter + tipos

```

1 import { EventEmitter } from 'node:events';
2
3 type EventosPedido = {
4   criado: [pedido: { id: string; total: number }];
5   cancelado: [id: string, motivo: string];
6 };
7
8 class EmissorTipado<E extends Record<string, unknown[]>> {
9   #emitter = new EventEmitter();
10
11   on<K extends keyof E & string>(evento: K, ouvinte: (...args: E[K]) => void) {
12     this.#emitter.on(evento, ouvinte);
13     return this;
14   }
15   emit<K extends keyof E & string>(evento: K, ...args: E[K]) {
16     return this.#emitter.emit(evento, ...args);
17   }
18 }
19
20 const pedidos = new EmissorTipado<EventosPedido>();
21 pedidos.on('cancelado', (id, motivo) => log(id, motivo)); // id: string,
22   ↪ motivo: string
23 pedidos.emit('criado', { id: 'p1', total: 99 }); // ok
24 pedidos.emit('criado', 'p1'); // erro de
25   ↪ compilação

```

- **TypeScript:** nomes de eventos e argumentos são checados via `keyof E` e `E[K]`. Versões recentes de `@types/node` também aceitam `EventEmitter<Mapa>` diretamente.
- **Runtime síncrono:** `emit` chama os listeners na hora, na mesma call stack.
- **JavaScript:** um evento `'error'` emitido sem listener derruba o processo.

7.7 Callbacks + Generics

```

1 function debounce<A extends unknown[]>(fn: (...args: A) => void, ms: number) {
2   let timer: NodeJS.Timeout | undefined;           // estado na closure
3   return (...args: A) => {
4     clearTimeout(timer);
5     timer = setTimeout(() => fn(...args), ms);
6   };
7 }
8 const salvarRascunho = debounce((id: string, texto: string) => { /* ... */ },
9   ↪ 300);
10 salvarRascunho('doc-1', 'olá'); // ok
11 salvarRascunho('doc-1');         // erro: esperava 2 argumentos
12
13 function agruparPor<T, K extends PropertyKey>(itens: T[], chave: (item: T) =>
14   ↪ K) {
15   const grupos = {} as Record<K, T[]>;
16   for (const item of itens) (grupos[chave(item)] ??= []).push(item);
17   return grupos;
18 }
19 const porStatus = agruparPor(pedidos, (p) => p.status); // Record<Status,
20   ↪ Pedido[]>

```

- **TypeScript:** A captura a assinatura do callback como tupla; `K` vira as chaves do resultado.
- **Closure:** mantém o `timer` entre chamadas do `debounce`.
- **JavaScript:** os timers rodam no Event Loop; o objeto `grupos` é construído em runtime.

7.8 Quando usar X ou Y

Dilema	Recomendação
<code>interface</code> ou <code>type</code>	<code>interface</code> para objetos extensíveis; <code>type</code> para unions e composições
<code>enum</code> ou union literal	prefira union literal (ou objeto <code>as const</code>); <code>enum</code> só se precisar de reverse mapping
<code>any</code> ou <code>unknown</code>	sempre <code>unknown</code> ; <code>any</code> apenas em migração
<code>as</code> ou type guard	type guard (ou schema) sempre que possível
<code>Pick/Omit</code> ou mapped type manual	utility types primeiro
Overload ou generic	generic quando a saída segue a entrada; overload quando as formas são realmente distintas
<code>private</code> ou <code>#campo</code>	<code>#campo</code> quando a privacidade precisa valer em runtime

7.9 Anti-patterns comuns

- `as` desnecessário, especialmente `as unknown as T`;
- `any` escondido em tipos de bibliotecas, em `JSON.parse` ou em `catch`;
- tipos complexos que ninguém do time entende;
- tipos duplicados que poderiam ser derivados com utility types ou `typeof`;
- uso excessivo de `!` (non-null assertion);
- confiar em tipos para dados externos sem validação em runtime;
- promises sem `await` e sem `.catch` (*floating promises*).

Chapter 8

Projetos práticos por nível

Um projeto real vale mais que dez tutoriais. Faça estes projetos em paralelo ao estudo; cada nível tem um critério objetivo de conclusão.

Nível 1 — Fundamentos

Objetivo: consolidar sintaxe, tipos básicos e estrutura de projeto.

- CLI tipada em Node.js + TS (por exemplo, um gerenciador de tarefas persistido em arquivo JSON);
- biblioteca de utilitários publicável, com `.d.ts` gerado.

Critério de conclusão: o projeto roda sem `any`, com `strict: true` e build limpo.

Nível 2 — Intermediário

Objetivo: integrar a tipagem com o mundo real (runtime, rede e persistência).

- API REST com Fastify + Zod + Prisma;
- sistema de eventos tipado com EventEmitter;
- wrapper de HTTP client com generics, timeout e retry.

Critério de conclusão: validação de input em runtime e testes de integração passando.

Nível 3 — Avançado

Objetivo: usar TypeScript como linguagem de design, não apenas como anotação.

- framework interno com decorators e injeção de dependências (um mini-NestJS);
- biblioteca com API em nível de tipos (por exemplo, um query builder tipado);
- monorepo com project references.

Critério de conclusão: API pública documentada pelos tipos e type tests cobrindo os principais fluxos.

Chapter 9

Node.js que vale estudar depois

Complemento essencial para quem faz backend. Estes assuntos rendem muito mais do que detalhes obscuros do JavaScript de browser. Os itens 1 a 10 formam o núcleo; 11 a 18 são a camada de produção.

#	Tópico	Por que importa	Sinal de domínio
1	Event Loop	latência, travamentos, ordem de execução	explica um p99 alto causado por código síncrono
2	EventEmitter	base de streams, sockets e HTTP	sabe que <code>emit</code> é síncrono e trata <code>'error'</code>
3	Streams	arquivos grandes, uploads, CSV	usa <code>pipeline()</code> e entende backpressure
4	Buffers	binários, encoding, protocolos	converte Buffer, string e base64 sem corromper
5	libuv	como o I/O assíncrono acontece	separa I/O de rede (SO) de fs/crypto (pool)
6	Thread Pool	gargalos em <code>bcrypt</code> , <code>fs</code> , <code>dns.lookup</code>	sabe quando ajustar <code>UV_THREADPOOL_SIZE</code>
7	Worker Threads	CPU-bound sem bloquear o servidor	move um cálculo pesado para um worker e mede
8	Child Processes	executar binários externos	escolhe entre <code>spawn</code> , <code>exec</code> e <code>fork</code>
9	Módulos ESM/CJS	interoperabilidade e pacotes	resolve <code>ERR_REQUIRE_ESM</code> , configura <code>exports</code>
10	Tratamento de erros	resiliência	trata <code>unhandledRejection</code> e <code>uncaughtException</code>
11	Cluster	usar todos os núcleos	compara cluster, PM2 e réplicas de container
12	Graceful shutdown	deploys sem derrubar requisições	trata <code>SIGTERM/SIGINT</code> e drena conexões
13	HTTP nativo	entender o que o framework esconde	sobe um servidor com <code>node:http</code> puro
14	<code>AsyncLocalStorage</code>	contexto por requisição	propaga request id sem passar parâmetro
15	<code>async_hooks</code> , <code>diagnostics_channel</code>	instrumentação de baixo nível	sabe como APMs se conectam ao Node
16	Observabilidade	logs, métricas e traces	instrumenta com OpenTelemetry
17	Segurança	OWASP básico	helmet, rate limit, validação de input
18	Deploy	colocar em produção	Docker, PM2, systemd

9.1 Streams com backpressure

```

1 import { createReadStream, createWriteStream } from 'node:fs';
2 import { pipeline } from 'node:stream/promises';
3 import { Transform } from 'node:stream';
4 import { createGzip } from 'node:zlib';
5
6 const maiusculas = new Transform({

```

```

7   transform(chunk: Buffer, _encoding, callback) {
8     callback(null, chunk.toString('utf8').toUpperCase());
9   },
10  });
11
12  await pipeline(                                // propaga erros e respeita backpressure
13    createReadStream('entrada.log'),
14    maiusculas,
15    createGzip(),                                // zlib roda no thread pool
16    createWriteStream('saida.log.gz'),
17  );

```

Detalhe do exemplo

Um chunk pode cortar um caractere UTF-8 multibyte no meio. Em produção, use `StringDecoder` ou `setEncoding('utf8')` na origem — exatamente o tipo de detalhe que o item 4 (Buffers) ensina.

9.2 Contexto por requisição e graceful shutdown

```

1  import { AsyncLocalStorage } from 'node:async_hooks';
2
3  const contexto = new AsyncLocalStorage<{ requestId: string }>();
4
5  app.addHook('onRequest', (req, _reply, done) => {
6    contexto.run({ requestId: req.id }, done);    // tudo o que roda depois herda
        ↪ o store
7  });
8
9  function log(msg: string) {
10    console.log(`[${contexto.getStore()?.requestId ?? '-'}] ${msg}`);
11  }
12
13  async function encerrar(sinal: NodeJS.Signals) {
14    log(`${sinal} recebido, encerrando`);
15    await app.close();                          // para de aceitar conexões e espera as em
        ↪ andamento
16    await db.$disconnect();
17    process.exit(0);
18  }
19  process.once('SIGTERM', encerrar);
20  process.once('SIGINT', encerrar);
21  process.on('unhandledRejection', (err) => { logger.fatal(err); process.exit(1);
        ↪ });

```

Chapter 10

Ordem recomendada de estudo

Leia de cima para baixo, a coluna da esquerda e depois a da direita. Voltar a um passo anterior quando um conceito novo expõe uma lacuna faz parte do processo.

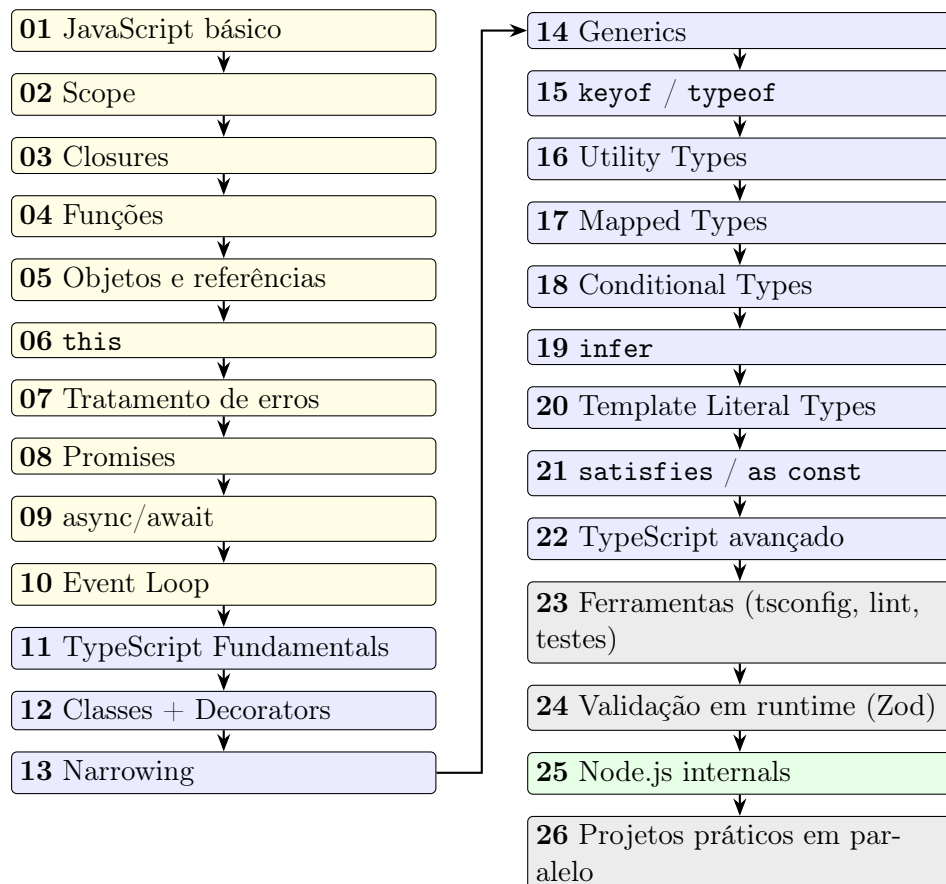


Figure 10.1: Sequência de estudo. Amarelo: JavaScript. Azul: TypeScript. Cinza: ferramentas e prática. Verde: Node.js.

Em paralelo, não em série

Os passos 01–10 não precisam terminar antes do 11. Uma cadência que funciona: em cada sessão de estudo, um tópico TS novo mais a revisão do tópico JS que ele usa (tabela do Capítulo 2). Faça rápido o que já sabe e gaste tempo real em closures, `this` e Event Loop. O passo 26 começa no primeiro dia.

Chapter 11

Checklist de domínio

Critérios observáveis: não “estudei”, mas “consigo fazer sem consultar”. Imprima este capítulo e revise mensalmente.

11.1 JavaScript

Tópico	Você domina quando consegue...	Seção
☐ Scope	explicar TDZ e a diferença entre <code>let</code> , <code>const</code> e <code>var</code>	3.1
☐ Closures	explicar o 3 3 3 do loop com <code>var</code> e desenhar o ambiente capturado	3.2
☐ Factory Functions	implementar uma factory com estado privado sem hesitar	3.2
☐ <code>this</code>	prever o valor de <code>this</code> em qualquer call site	3.3
☐ Arrow Functions	dizer quando <i>não</i> usar arrow	3.3
☐ Objects / References	encontrar uma mutação acidental por referência compartilhada	3.4
☐ Destructuring	desestruturar aninhado com renomeação, default e rest	3.4
☐ Spread / Rest	distinguir cópia rasa de profunda e usar rest em parâmetros	3.5
☐ Array methods	usar <code>reduce</code> sem consultar e saber quais métodos mutam	3.6
☐ Tratamento de erros	criar erros customizados corretamente, com <code>cause</code>	3.7
☐ <code>?.</code> e <code>??</code>	escolher entre <code>??</code> e <code> </code> pelo significado de 0 e <code>''</code>	3.8
☐ Promises	usar <code>allSettled</code> quando apropriado	3.9
☐ Async/Await	paralelizar awaits independentes e tratar erros corretamente	3.10
☐ Event Loop	explicar a ordem de micro e macrotasks	3.11
☐ Prototype	desenhar a prototype chain de uma hierarquia de classes	3.12
☐ Map / Set / WeakMap / WeakSet	justificar cada um no lugar de objetos e arrays	3.13
☐ ESM / CommonJS	configurar ESM do zero e explicar um ciclo de dependência	3.13

11.2 TypeScript

Tópico	Você domina quando consegue...	Seção
☐ Unions / Intersections	modelar estados como union de literais e compor com &	4.1
☐ Classes + Decorators	escrever hierarquias com <code>abstract/override</code> e um decorator simples	4.2
☐ Narrowing	usar <code>typeof</code> , <code>in</code> , <code>instanceof</code> e igualdade sem assertions	4.3
☐ Type Guards	escrever predicates corretos e discriminated unions	4.3
☐ Assertion Functions	escrever <code>asserts x is T</code> para validar pré-condições	4.3
☐ Generics	escrever constraints sem consultar	4.4
☐ <code>keyof</code> , <code>typeof</code> , indexed access	derivar tipos de valores e diferenciar do <code>typeof</code> de runtime	4.6
☐ Utility Types	derivar DTOs de criação, atualização e resposta de um único tipo	4.5
☐ Mapped Types	reescrever <code>Partial</code> e usar key remapping	4.7
☐ Conditional Types	explicar distributividade e reimplementar <code>Exclude</code>	4.8
☐ <code>infer</code>	reimplementar <code>ReturnType</code> e <code>Awaited</code>	4.9
☐ Template Literal Types	tipar parâmetros de rota ou <code>Path<T></code>	4.10
☐ <code>satisfies</code>	saber quando preferir <code>satisfies</code> , <code>as const</code> ou anotação	4.11
☐ <code>unknown</code> / <code>never</code>	validar entradas externas e usar exhaustive checks	4.12
☐ Function Overloads	decidir entre overload, union e generic	4.14
☐ Structural Typing	explicar excess property checks e usar branded types	5.1
☐ Variance	explicar por que parâmetros são contravariantes	5
☐ <code>.d.ts</code>	escrever declarações próprias para uma lib sem tipos	5
☐ Module Augmentation	adicionar campos ao request do seu framework	5
☐ Advanced <code>tsconfig</code>	configurar um projeto strict do zero	5.8
☐ Type tests	escrever testes de tipos com <code>expect-type</code> ou Vitest	6

11.3 Ferramentas e ecossistema

☐ Configuro ESLint (<code>typescript-eslint type-aware</code>) + Prettier + TS em um projeto novo
☐ Escrevo testes com Vitest ou Jest, com mocks tipados
☐ Valido input HTTP e variáveis de ambiente com Zod
☐ Configuro um ORM tipado (Prisma ou Drizzle) com migrations
☐ Faço build com <code>tsup</code> ou <code>esbuild</code> e checo tipos com <code>tsc -noEmit</code> no CI

11.4 Node.js

-
- ☐ Diagnostico bloqueio do Event Loop em um serviço real
 - ☐ Processo arquivos grandes com Streams e `pipeline()`, respeitando backpressure
 - ☐ Movo trabalho CPU-bound para Worker Threads
 - ☐ Implemento graceful shutdown com `SIGTERM`
 - ☐ Propago contexto de requisição com `AsyncLocalStorage`
 - ☐ Tenho logs estruturados, métricas e traces básicos (observabilidade)
-

Teste final de domínio

Pegue um módulo real do seu backend (controller + service + repositório) e: (1) remova todo `any`; (2) derive os DTOs com utility types; (3) valide o input com um schema; (4) modele estados com discriminated union e exhaustive check; (5) explique, em voz alta e linha a linha, o que acontece em runtime quando uma requisição chega.

Chapter 12

Regras do roadmap

1. **Não tente terminar JavaScript antes de começar TypeScript.**
JavaScript não tem fim. Se esperar dominá-lo por completo, você nunca começa o que realmente queria aprender.
2. **Aprenda TypeScript e aprofunde JavaScript conforme os conceitos aparecerem.**
Cada dúvida de runtime que surge no TS indica qual tópico de JS estudar agora.
3. **O objetivo não é decorar APIs, mas entender por que o código funciona.**
A documentação resolve a assinatura de `reduce`. Ela não resolve a dúvida sobre por que o `this` sumiu.
4. **TypeScript ajuda em compile time; JavaScript determina o comportamento em runtime.**
Tipos descrevem expectativas. Validação, closures, Promises e o Event Loop decidem o que acontece de fato.
5. **Se você não sabe explicar em voz alta, você não domina.**
Explicar para outra pessoa (ou para o pato de borracha) expõe as lacunas que a leitura esconde.
6. **Um projeto real vale mais que dez tutoriais concluídos.**
Tutoriais mostram o caminho feliz; projetos mostram os erros, as integrações e as decisões.

Appendix A

Referências e Leitura Adicional

Documentação oficial

- TypeScript Handbook — <https://www.typescriptlang.org/docs/handbook/>
- TSConfig Reference — <https://www.typescriptlang.org/tsconfig/>
- TypeScript Release Notes — <https://www.typescriptlang.org/docs/handbook/release-notes/overview.html>
- TypeScript Playground (veja o JS emitido ao lado do TS) — <https://www.typescriptlang.org/play>
- MDN Web Docs — JavaScript — <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- Node.js — The Node.js Event Loop — <https://nodejs.org/en/learn/asynchronous-work-event-loop-timers-and-nexttick>
- Node.js API Documentation — <https://nodejs.org/api/>

Livros

- Boris Cherny, *Programming TypeScript*, O'Reilly.
- Dan Vanderkam, *Effective TypeScript*, O'Reilly.
- Kyle Simpson, *You Don't Know JS Yet* (leia seletivamente: scope, closures, `this`, prototypes) — <https://github.com/getify/You-Dont-Know-JS>

Prática e comunidade

- type-challenges — <https://github.com/type-challenges/type-challenges>

- **tsconfig/bases** (configurações de referência) — <https://github.com/tsconfig/bases>
- **Total TypeScript** (Matt Pocock) — <https://www.totaltypescript.com>
- Newsletter **TypeScript Weekly**.
- Canais: Matt Pocock, Theo, Web Dev Simplified.

Appendix B

Glossário

Type erasure remoção das anotações de tipo na compilação; o JS gerado não contém tipos.

Lexical environment estrutura que mapeia nomes a valores em um escopo, com referência para o escopo externo.

Closure função que retém acesso ao escopo léxico onde foi criada.

Hoisting elevação de declarações ao topo do escopo antes da execução.

TDZ Temporal Dead Zone: período entre o início do bloco e a inicialização de `let/const`.

Factory function função que cria e retorna objetos, geralmente com estado privado via closure.

Call site o ponto onde uma função é chamada; determina o valor de `this`.

Shallow copy cópia que duplica apenas o primeiro nível de um objeto.

Prototype chain cadeia de objetos usada na resolução de propriedades.

Microtask / Macrotask filas distintas do Event Loop, com prioridades diferentes.

libuv biblioteca em C que implementa o Event Loop e o thread pool do Node.js.

Narrowing redução do tipo de um valor a partir de uma checagem em runtime.

Type guard função cujo retorno é `x is Y`.

Assertion function função com retorno `asserts x is Y`; lança erro se a condição falhar.

Discriminated union union de objetos com um campo literal comum que identifica cada variante.

Generics parâmetros de tipo que relacionam entradas e saídas.

Generic constraint restrição aplicada a um parâmetro de tipo via `extends`.

Utility types tipos da biblioteca padrão que transformam outros tipos.

Mapped type tipo derivado iterando sobre as chaves de outro tipo.

Conditional type tipo que resolve com base em `T extends U ? X : Y`.

Distributividade aplicação automática de um conditional type a cada membro de uma union.

infer extrai um tipo dentro de um conditional type.

Template literal type tipo de string construído com a sintaxe de template.

satisfies operador que valida a forma preservando o tipo inferido.

unknown tipo seguro para valores desconhecidos; exige narrowing antes do uso.

never tipo sem valores; usado em exhaustive checks.

Decorator função que modifica ou anota classes, métodos e propriedades.

Structural typing tipagem por forma, não por nome.

Variance como subtipos se relacionam em posições covariantes e contravariantes.

Branded type tipo nominal simulado por uma marca única.

Declaration merging fusão de declarações com o mesmo nome.

Module augmentation extensão de um módulo existente a partir de outro arquivo.

ESM / CommonJS os dois sistemas de módulos do Node.js.

tsconfig arquivo de configuração do compilador TypeScript.

Validação em runtime verificação de dados externos durante a execução, por exemplo com Zod.

Tratamento de erros estratégia para lançar, capturar, relançar e reportar falhas.

Appendix C

FAQ e erros comuns

Por que `any` é ruim se funciona?

Porque desliga o sistema de tipos para aquele valor — e para tudo o que deriva dele. O código compila, mas você perde as garantias que o TypeScript existe para dar. Use `unknown` quando não souber o tipo.

Quando usar `interface` e quando usar `type`?

`interface` para contratos de objeto que podem ser estendidos ou mesclados. `type` para unions, interseções, aliases de primitivos e tipos compostos.

Por que meu `this` está `undefined`?

Porque a função foi chamada isoladamente (não como método) e você está em strict mode — o caso típico é passar um método como callback. Use `bind` ou uma arrow function para preservar o `this` (Seção [3.3](#)).

Por que o Zod não substitui o TypeScript?

Zod valida em **runtime**; TypeScript checa em **compile time**. São complementares: o Zod verifica dados externos (HTTP, env, banco) e infere os tipos que o TS usa no resto do código.

Por que a Promise não é cancelável?

Porque uma Promise representa um resultado que já está sendo produzido. Para cancelar o trabalho, use `AbortController` e APIs que aceitam `signal` (`fetch`, `setTimeout` de `node:timers/promises`, `streams`).

Por que `Promise.all` rejeita tudo se uma falhar?

Porque ele é “tudo ou nada”. Para coletar sucessos e falhas individualmente, use `Promise.allSettled`.

Por que `Object.keys` retorna `string[]` e não `(keyof T)[]`?

Por causa da tipagem estrutural: um objeto do tipo `T` pode ter propriedades extras em runtime, então o TypeScript não pode garantir que todas as chaves sejam `keyof T` (Seção 4.6).

Por que o `tsc` passa, mas o `import` com alias quebra em runtime?

Porque `paths` só existe em compile time; o `tsc` não reescreve os imports no JS emitido (Seção 5.8).

Meu `tsx` roda o código, então os tipos estão certos?

Não. `tsx`, `esbuild`, `swc` e o type stripping do Node apenas apagam os tipos. Só o `tsc` checa (Capítulo 6).

Index

- branded types, 41, 70
- closure, 12, 69
- CommonJS, 22, 70
- conditional types, 33, 70
- declaration merging, 42, 70
- decorators, 26, 70
- discriminated union, 28, 69
- distributividade, 33, 70
- ESM, 22, 70
- Event Loop, 19, 69
- factory function, 12, 69
- função de ordem superior, 15
- generics, 29, 69
- hoisting, 11, 69
- infer, 34, 70
- lexical environment, 11, 69
- lexical scope, 11
- libuv, 19, 69
- mapped types, 32, 70
- module augmentation, 42, 70
- narrowing, 28, 69
- never, 36, 70
- Promise, 18
- prototype chain, 21, 69
- satisfies, 35, 70
- shallow copy, 14, 69
- structural typing, 39, 70
- TDZ, 11, 69
- template literal types, 34, 70
- this, 13, 69
- tratamento de erros, 16, 70
- tsconfig, 44, 70
- type erasure, 7, 69
- unknown, 36, 70
- utility types, 30, 69
- validação em runtime, 47, 70
- variance, 40, 70